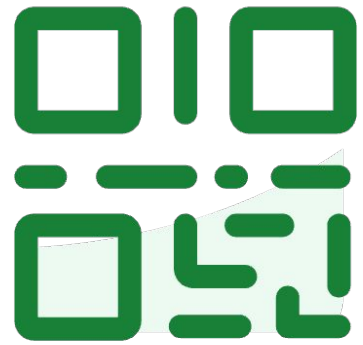


Do not edit
How to change the
design



**Join at slido.com
#3715895**

① The Slido app must be installed on every computer you're presenting from

slido

Course Evaluation



The final course survey is intended to help us improve future offerings of the course.

If you complete our **internal anonymous survey AND the official Course Evaluation** (<http://course-evaluations.berkeley.edu>) you will get **0.5% extra credit points** to your grade. If **90%** of the class also submits both, then everyone who **submitted both** will receive **an additional 0.5% percentage point** on their final grade.



Internal course survey link:
<https://bit.ly/cs189-eval>

Lecture 24

Self-Supervised Learning

Data understanding using self-supervised representation learning

EECS 189/289, Fall 2025 @ UC Berkeley

Joseph E. Gonzalez and Narges Norouzi

Roadmap

- Self-Supervised Learning
- Transfer Learning
- Self-Supervised Learning Approaches
- Generative
 - Autoencoders
 - Colorization
 - Cross-Channel Prediction
 - Context-Encoders (Inpainting)
 - Image Super-Resolution
- Discriminative
 - Rotation
 - Jigsaw
 - Clustering
 - Contrastive Learning



3715895

Self-Supervised Learning

- Self-Supervised Learning
- Transfer Learning
- Self-Supervised Learning Approaches
- Generative
 - Autoencoders
 - Colorization
 - Cross-Channel Prediction
 - Context-Encoders (Inpainting)
 - Image Super-Resolution
- Discriminative
 - Rotation
 - Jigsaw
 - Clustering
 - Contrastive Learning



3715895

Supervised vs. Unsupervised Learning



- *Supervised learning:*
learning with **labeled data**

- **Approach:** Collect a large dataset, label the data, train a model, and deploy.
- Costly

- *Unsupervised learning:*
learning with **unlabeled data**

- **Approach:** Discover patterns in data either via clustering similar instances, or density estimation, or dimensionality reduction ...

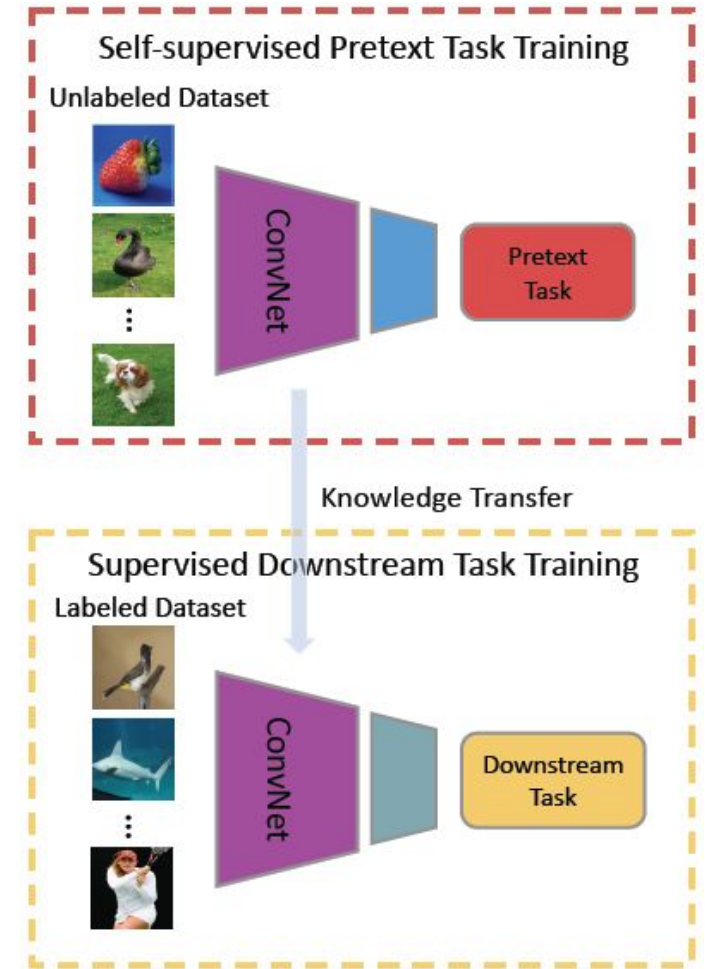
None of these represent how humans learn.

Let's build methods that learn from "raw" data with no annotations required.

Self-Supervised Learning



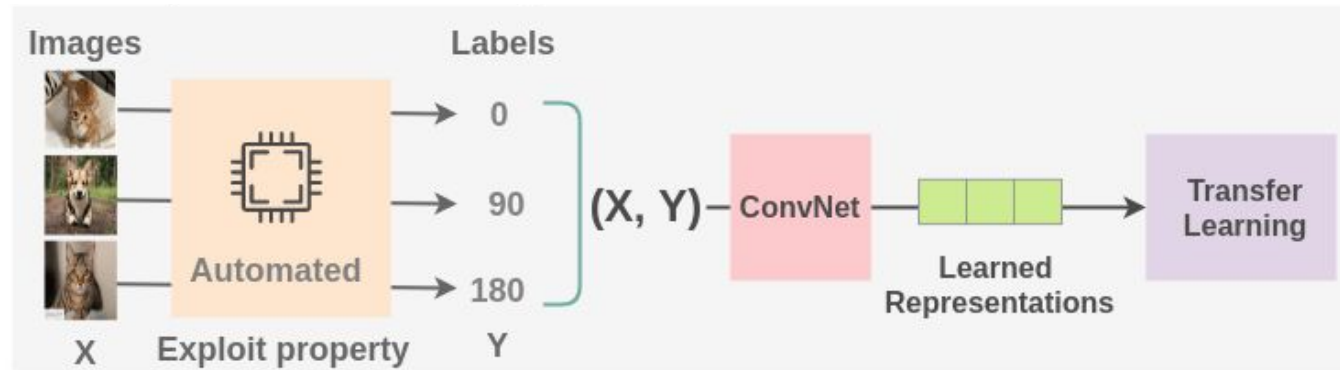
- **Unsupervised Learning:** Model isn't told what to predict.
- **Self-Supervised Learning:** Representation learning with **unlabeled data**
 - Learn useful **feature representations** from unlabeled data through **pre-text tasks**.
 - The term “self-supervised” refers to creating **its own supervision** (i.e., without supervision, without labels)



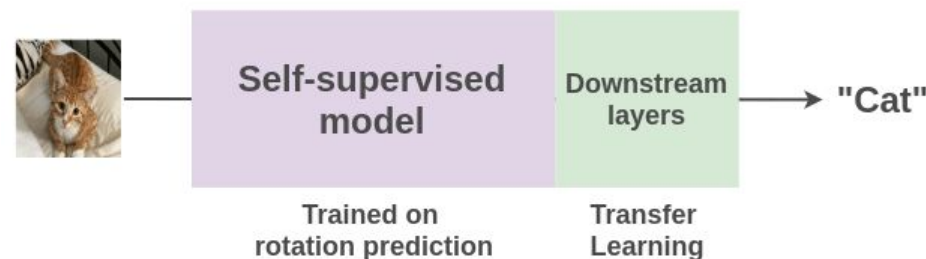
Self-Supervised Learning Example



- **Pre-text task**: Train a model to **predict the rotation degree** of rotated images with cats and dogs (can collect million of images, labeling is not required).



- **Downstream task**: Use transfer learning and fine-tune the learned model from the pre-text task for **classification** of cat vs. dog with very few labeled examples.



How? Transfer Learning

- Self-Supervised Learning
- **Transfer Learning**
- Self-Supervised Learning Approaches
- Generative
 - Autoencoders
 - Colorization
 - Cross-Channel Prediction
 - Context-Encoders (Inpainting)
 - Image Super-Resolution
- Discriminative
 - Rotation
 - Jigsaw
 - Clustering
 - Contrastive Learning

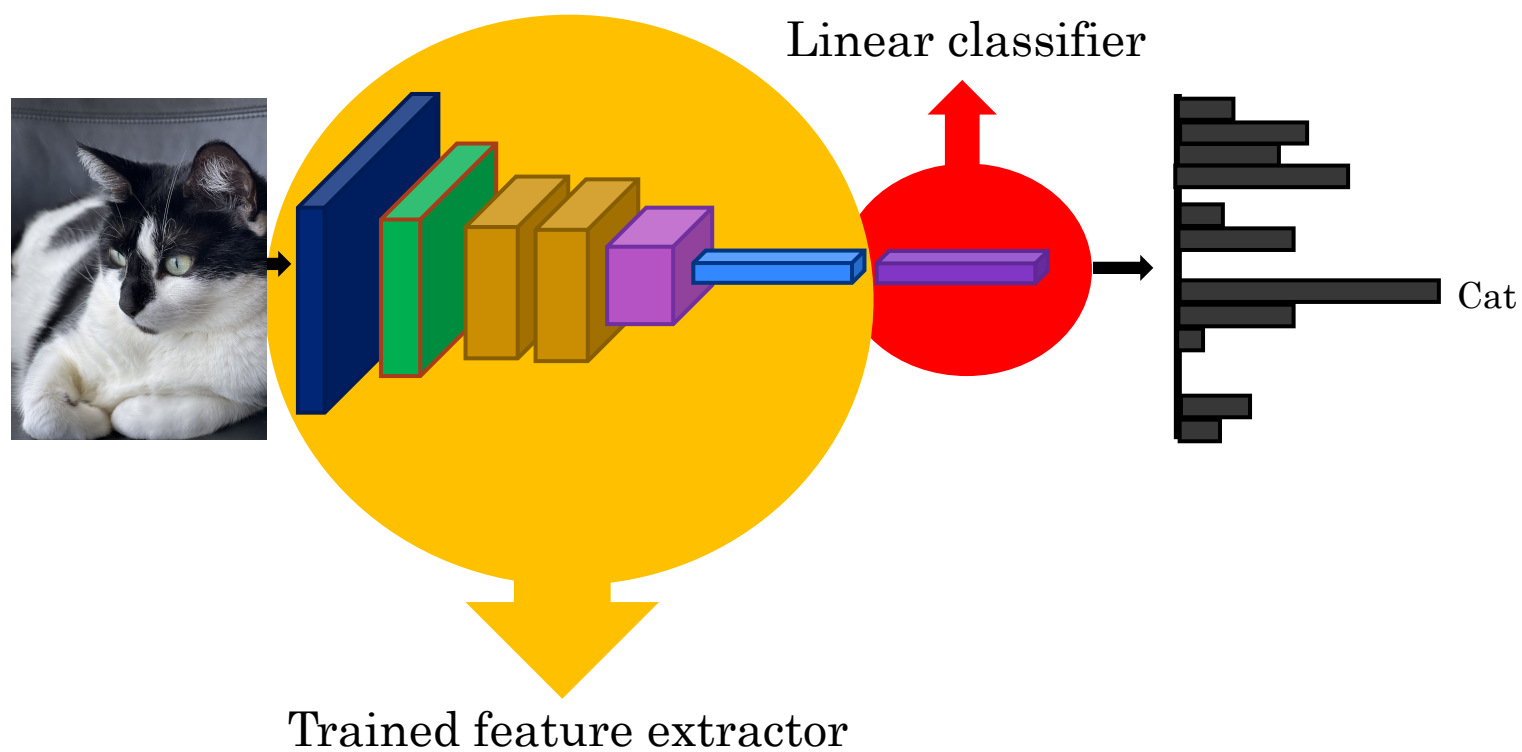




Transfer Learning

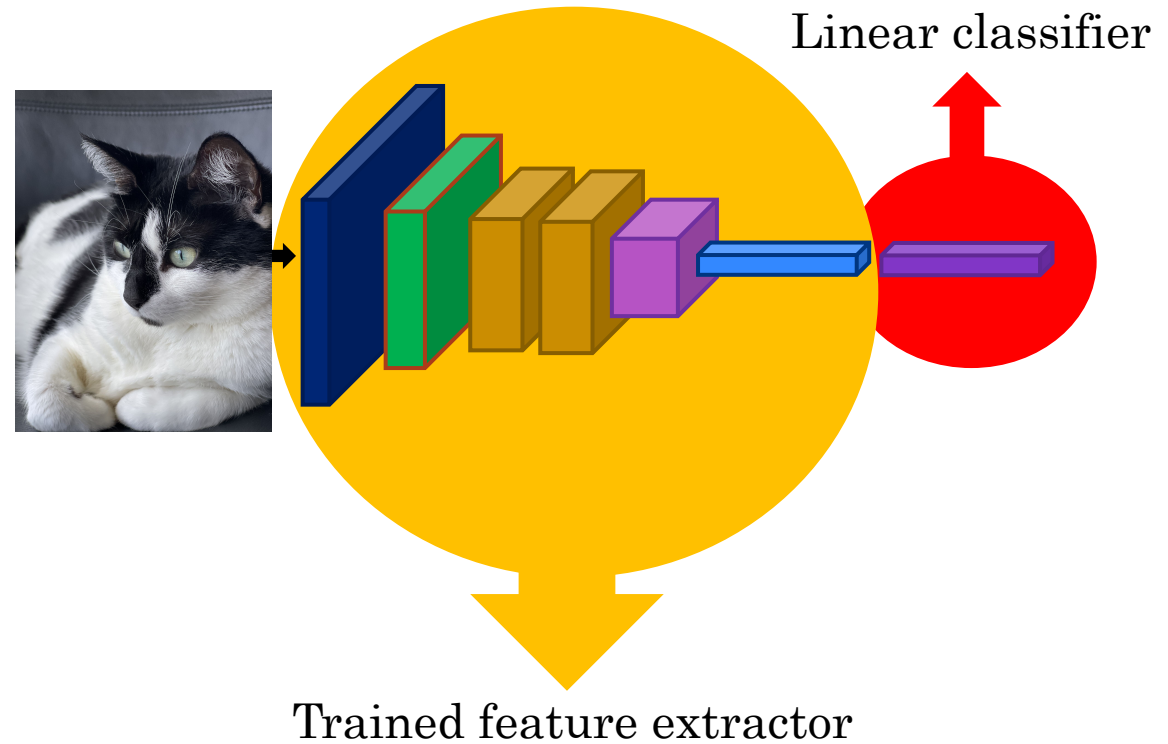
- A **shortcut** in training neural networks for recognition tasks.
- The idea is to start with a fully-trained image recognition neural network, off-the-shelf with trained weights.
- We can **re-purpose** the trained network for our particular recognition task.
- What was learned by the neural network in its early layers are useful features in recognizing various things in images.
- Most of the famous models (pre-trained or not pre-trained) exists on TorchVision: <https://docs.pytorch.org/vision/main/models.html>

Transfer Learning with CNN



Transfer Learning with CNN

- What do we do for a new image classification problem?
- Key idea:
 - *Freeze* parameters in feature extractor
 - *Re-train* classifier





Do not edit
How to change the
design



Given a small labeled dataset for a new classification task, what is the most effective initial approach to leverage a pretrained CNN?

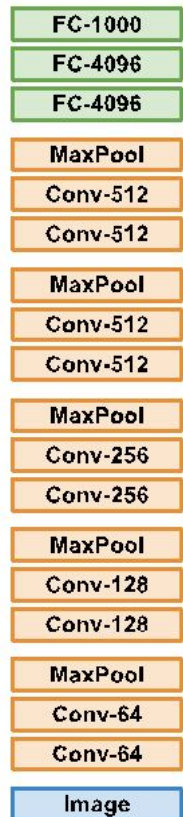
① The Slido app must be installed on every computer you're presenting from

slido

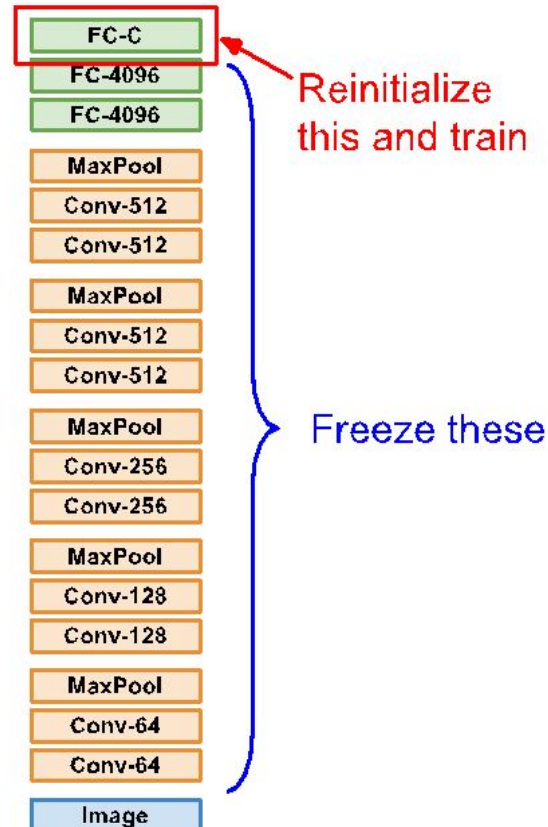
Transfer Learning With CNN



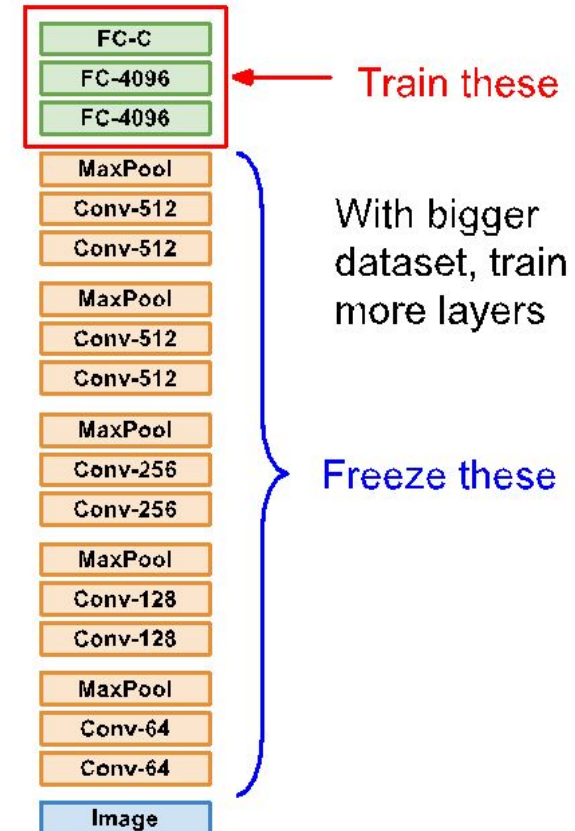
1. Train on Imagenet



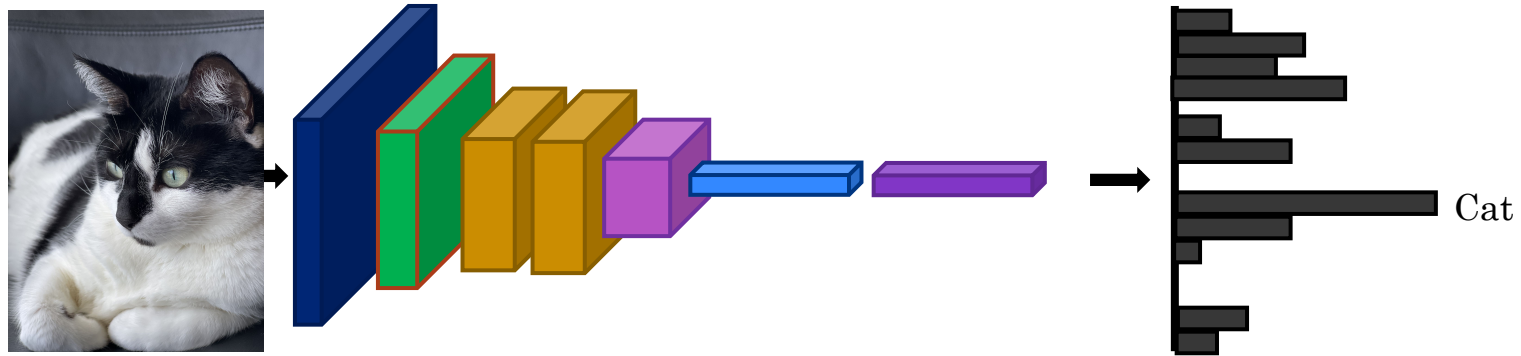
2. Small Dataset (C classes)



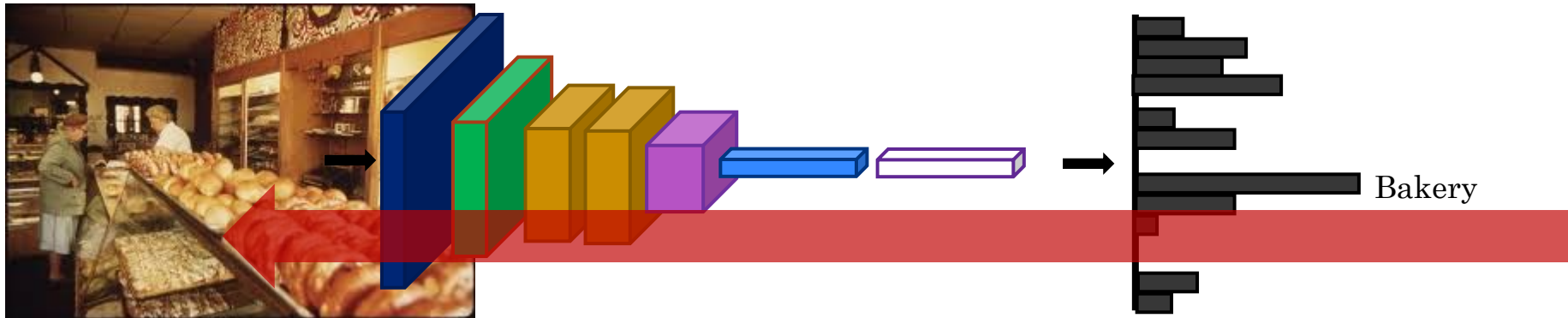
3. Bigger dataset



Fine-tuning



Fine-tuning

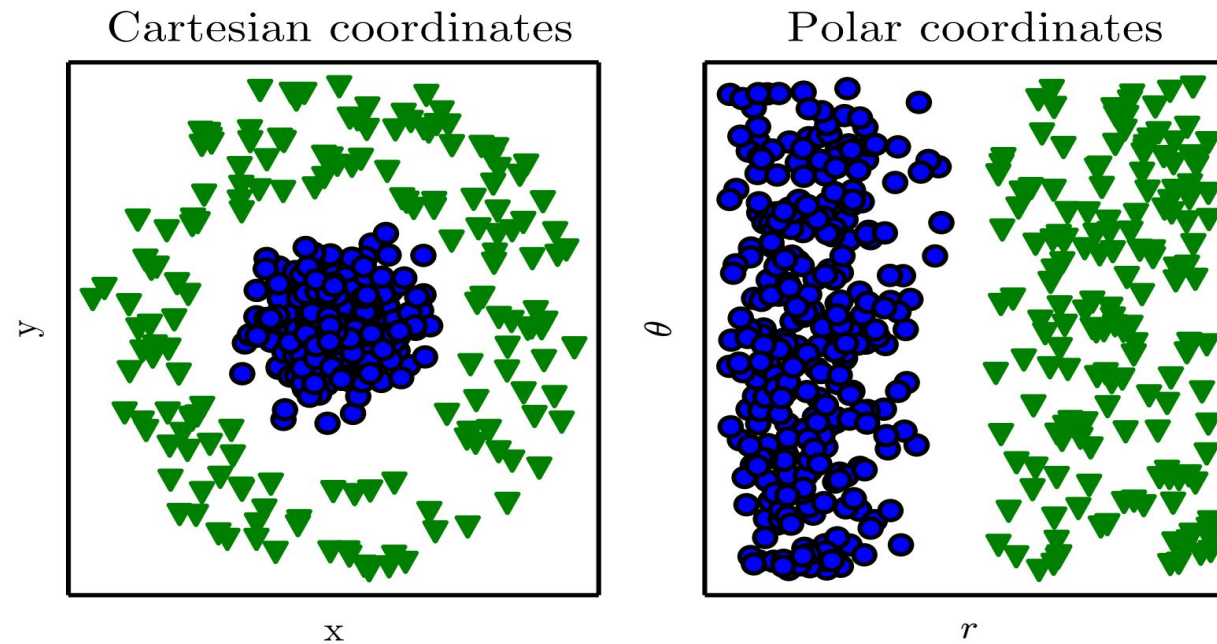


Initialize with
pre-trained, then train
with low learning rate

Why Pretext Learning Is Important



Representations Matter



Goodfellow



Self-Supervised Learning Challenges

- *Challenges* for self-supervised learning
 - How to select a suitable pre-text task for an application?
 - There is no gold standard for comparison of learned feature representations.
 - Selecting a suitable loss functions, since there is no single objective as the test set accuracy in supervised learning.

Self-Supervised Learning Approaches

- Self-Supervised Learning
- Transfer Learning
- Self-Supervised Learning Approaches
- Generative
 - Autoencoders
 - Colorization
 - Cross-Channel Prediction
 - Context-Encoders (Inpainting)
 - Image Super-Resolution
- Discriminative
 - Rotation
 - Jigsaw
 - Clustering
 - Contrastive Learning





Approaches

Generative: Predict part of the input signal

- Autoencoders
- GANs
 - Super-resolution
- Colorization
- Inpainting

Discriminative: Predict something about the input signal

- Context Prediction
- Rotation
- Jigsaw
- Clustering
- Contrastive

Multimodal: Use some additional signal in addition to RGB images.

- Video
- 3D
- Sound
- Language

Generative: Autoencoders

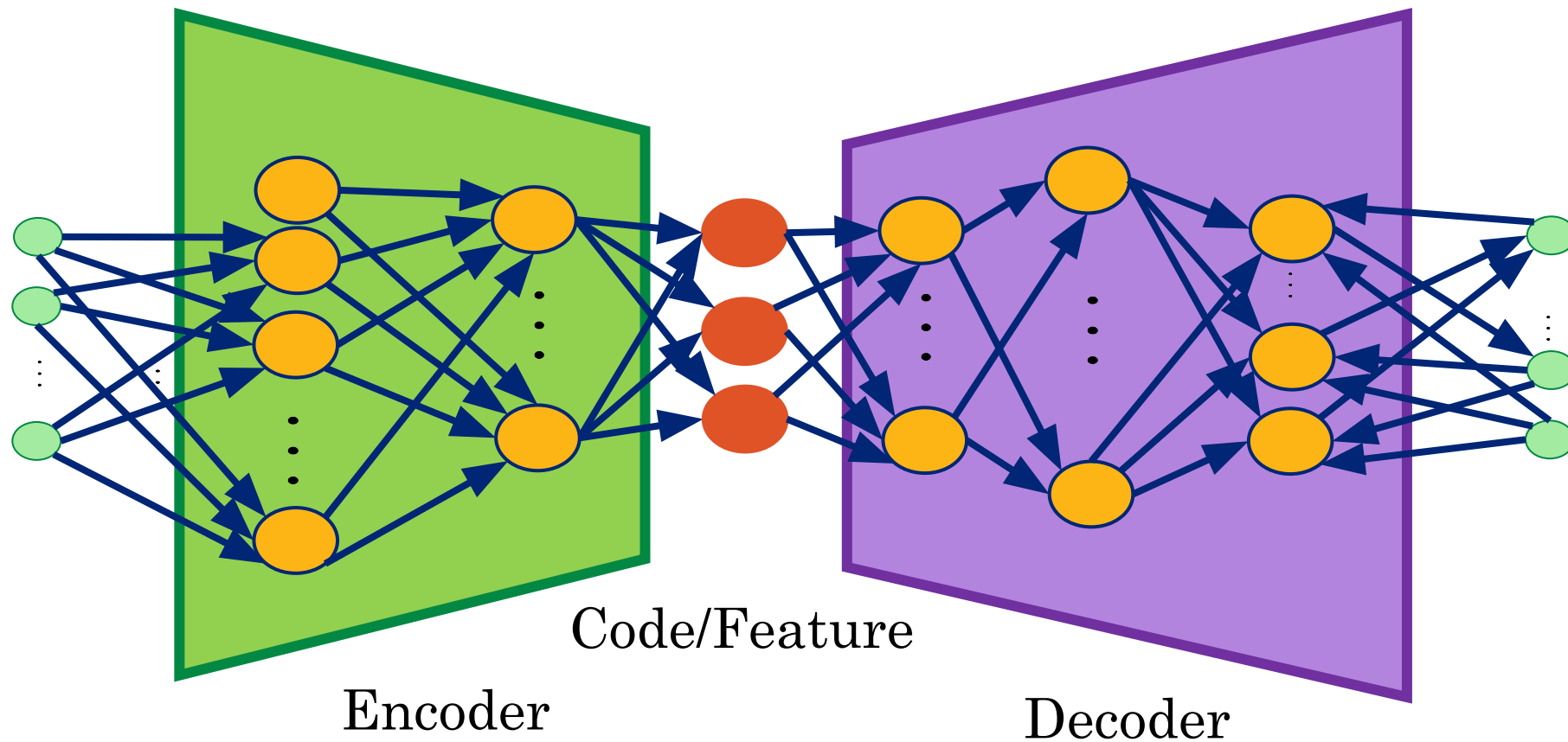
- Self-Supervised Learning
- Transfer Learning
- Self-Supervised Learning Approaches
- Generative
 - Autoencoders
 - Colorization
 - Cross-Channel Prediction
 - Context-Encoders (Inpainting)
 - Image Super-Resolution
- Discriminative
 - Rotation
 - Jigsaw
 - Clustering
 - Contrastive Learning



Autoencoders



- Autoencoders are designed to reproduce their input, especially for images.
 - Key point is to reproduce the input from a learned encoding.



Autoencoder Idea

Given data $\mathbf{x}$ (no labels) we would like to learn the functions f (encoder) and g (decoder) where:

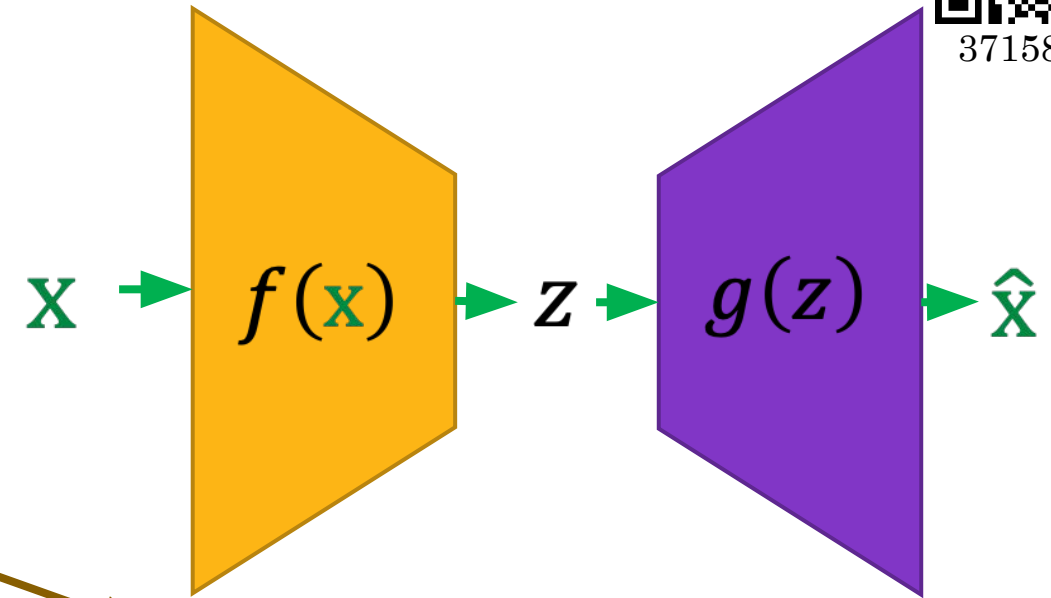
$$f(\mathbf{x}) = s(w\mathbf{x} + b) = \mathbf{z}$$

and

$$g(\mathbf{z}) = s(w'\mathbf{z} + b') = \hat{\mathbf{x}}$$

$$\text{s.t } h(\mathbf{x}) = g(f(\mathbf{x})) = \hat{\mathbf{x}}$$

where h is an **approximation** of the identity function.



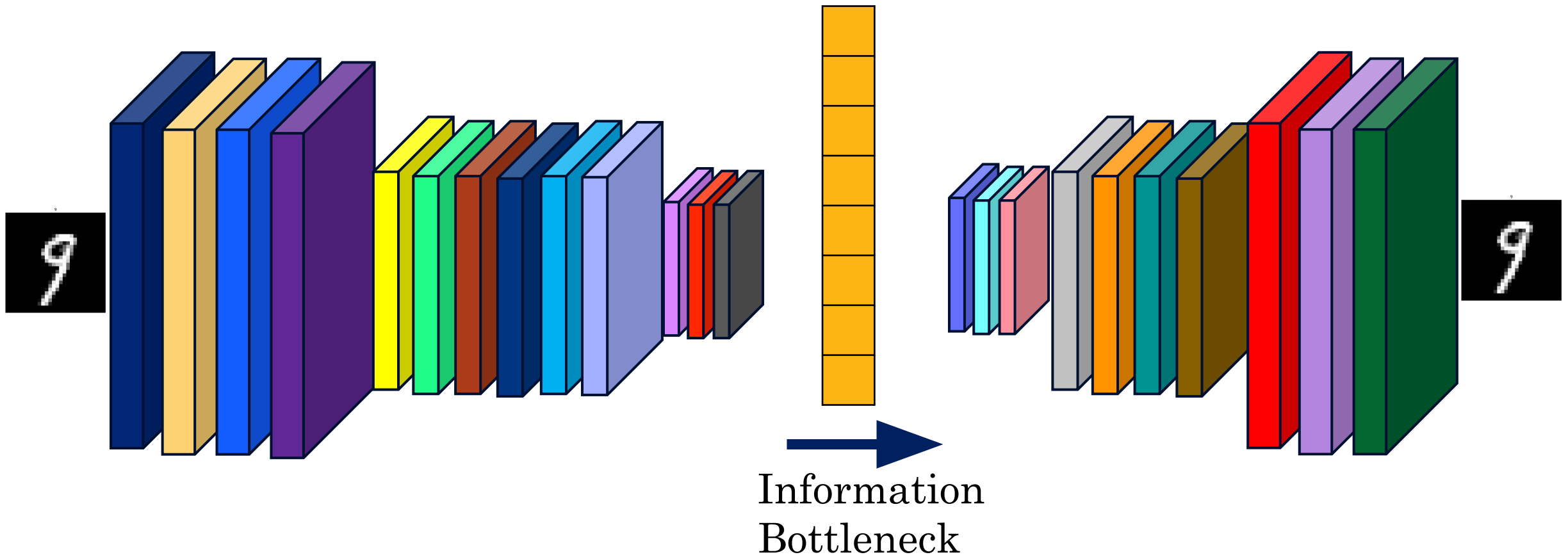
($\mathbf{z}$ is some **latent** representation or **code** and s is a non-linearity such as the sigmoid)

($\hat{\mathbf{x}}$ is $\mathbf{x}$'s reconstruction)



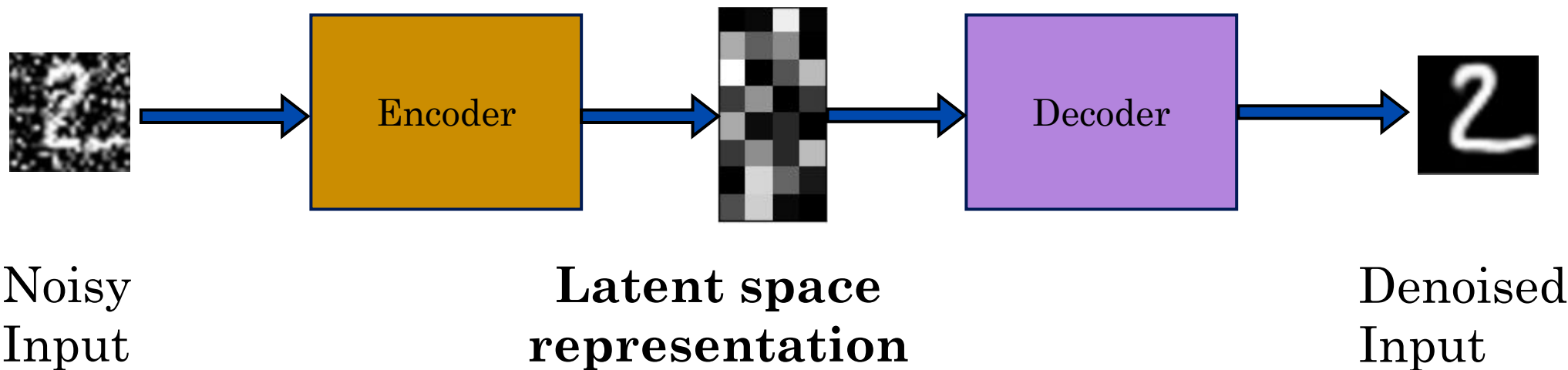
3715895

Convolutional Autoencoder



Denoising Autoencoders

- We try to undo the effect of *corruption* process stochastically applied to the input.
- We still aim to encode the input and to NOT mimic the identity function.



Denoising Autoencoders

Idea: A robust representation against noise:

- Random assignment of subset of inputs to 0, with probability v .
- Gaussian additive noise.

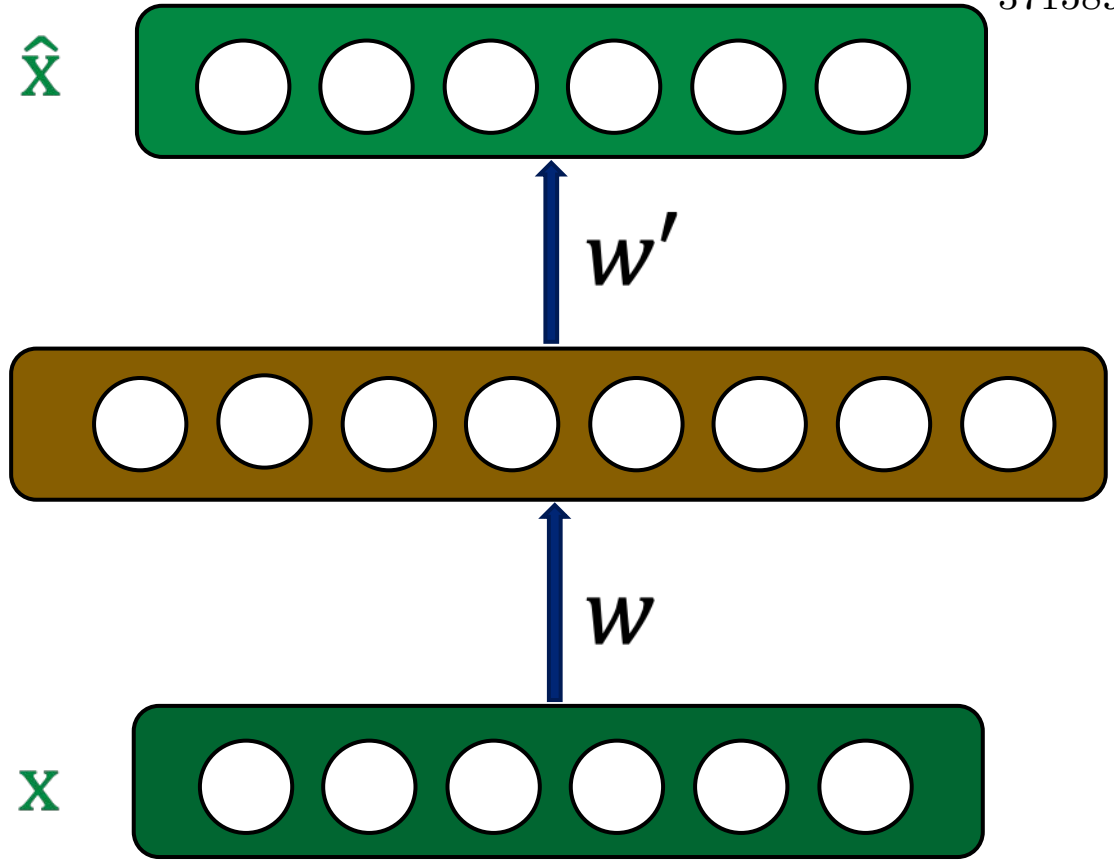
$f(\mathbf{x})$



(a)



(b)



3715895

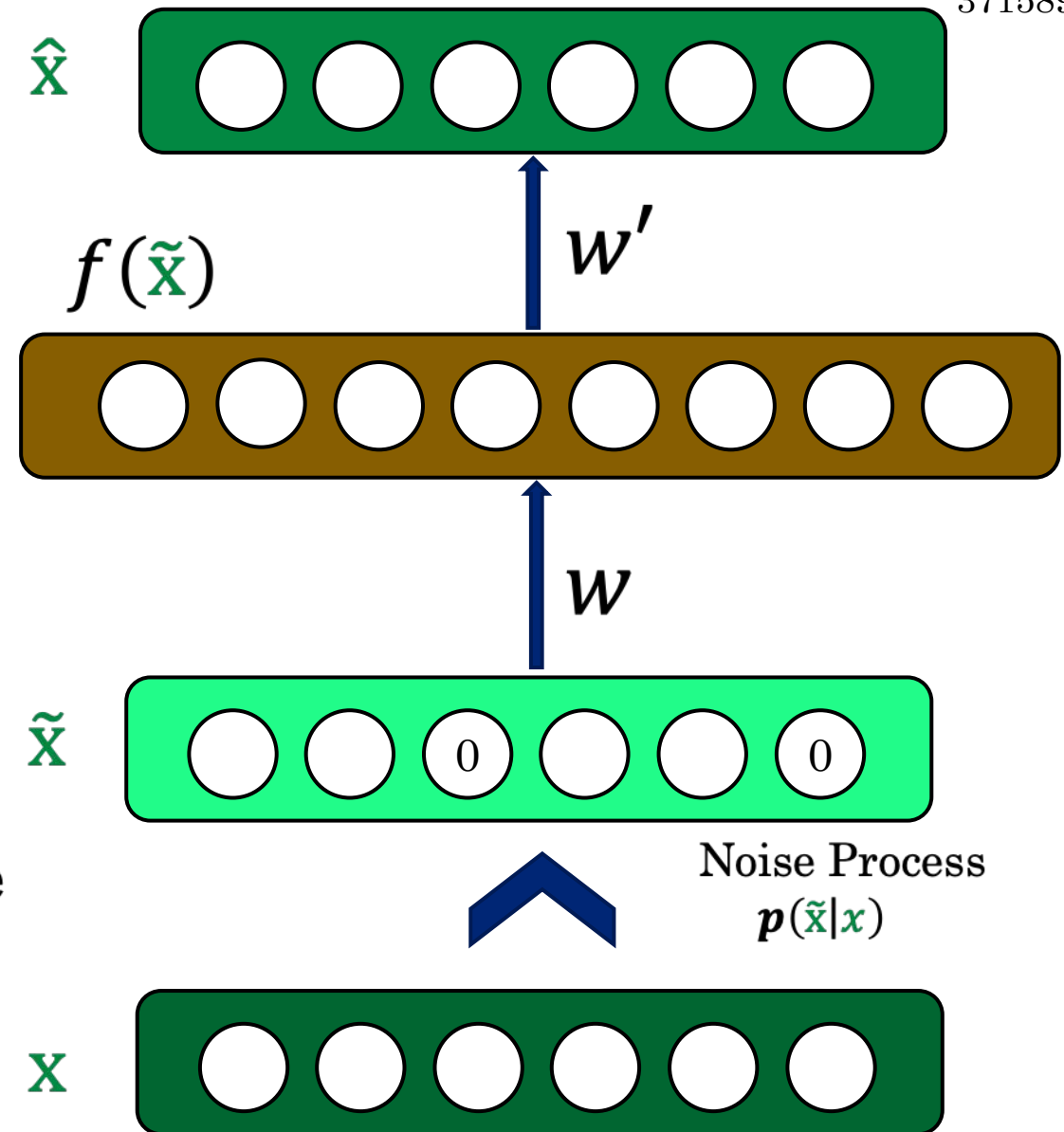


What are valid training objectives for a denoising auto encoder?



3715895

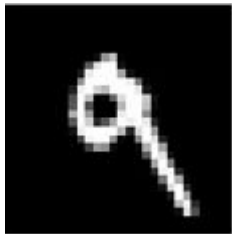
- Reconstruction $\hat{\mathbf{x}}$ computed from the corrupted input $\tilde{\mathbf{x}}$.
- Loss function compares $\hat{\mathbf{x}}$ reconstruction with the noiseless $\mathbf{x}$.
- The autoencoder cannot fully trust each feature of $\mathbf{x}$ independently so it must learn from data patterns.
- We are forcing the hidden layer to learn a generalized structure of the data.



Denoising Autoencoders - Process



Taken some input $\mathbf{x}$



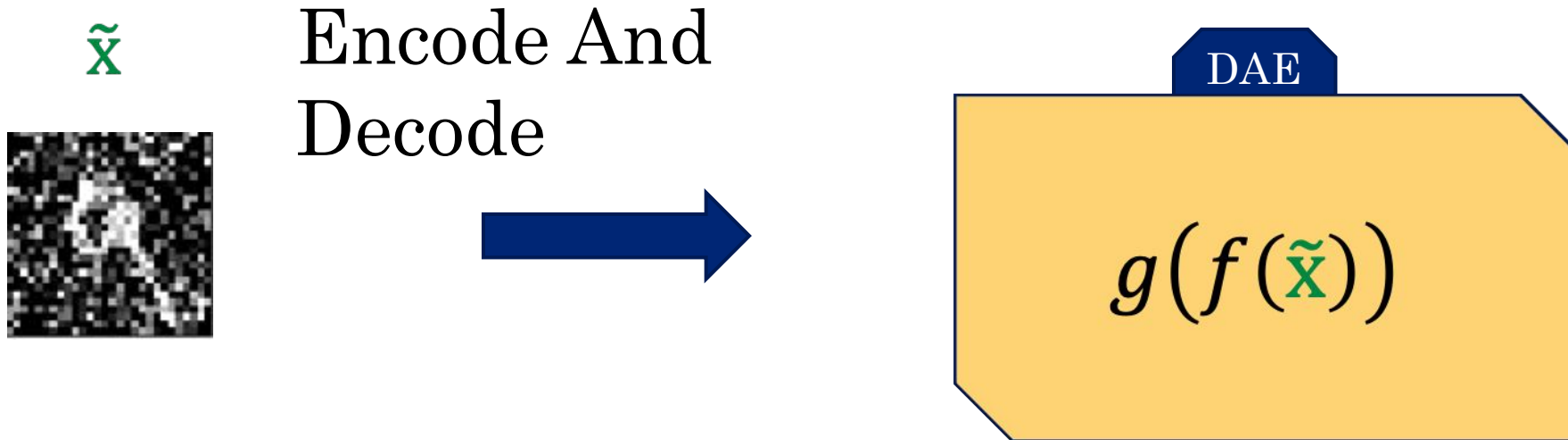
Apply
Noise



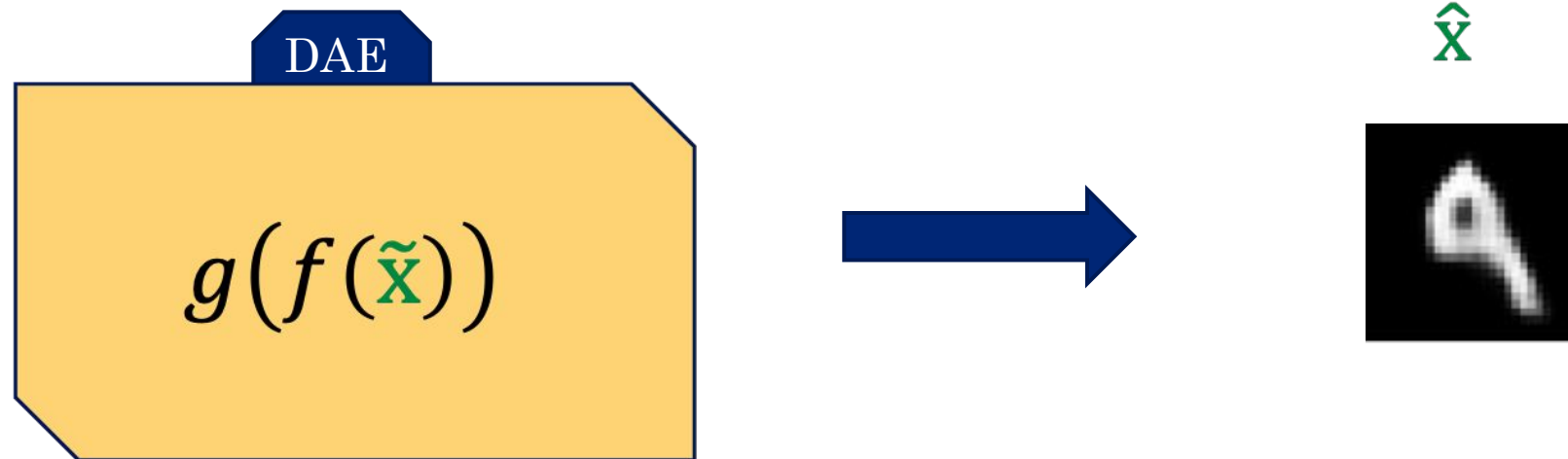
$\tilde{\mathbf{x}}$



Denoising Autoencoders - Process



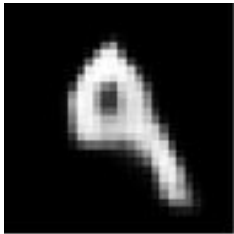
Denoising Autoencoders - Process



Denoising Autoencoders - Process



$\hat{x}$



Compare

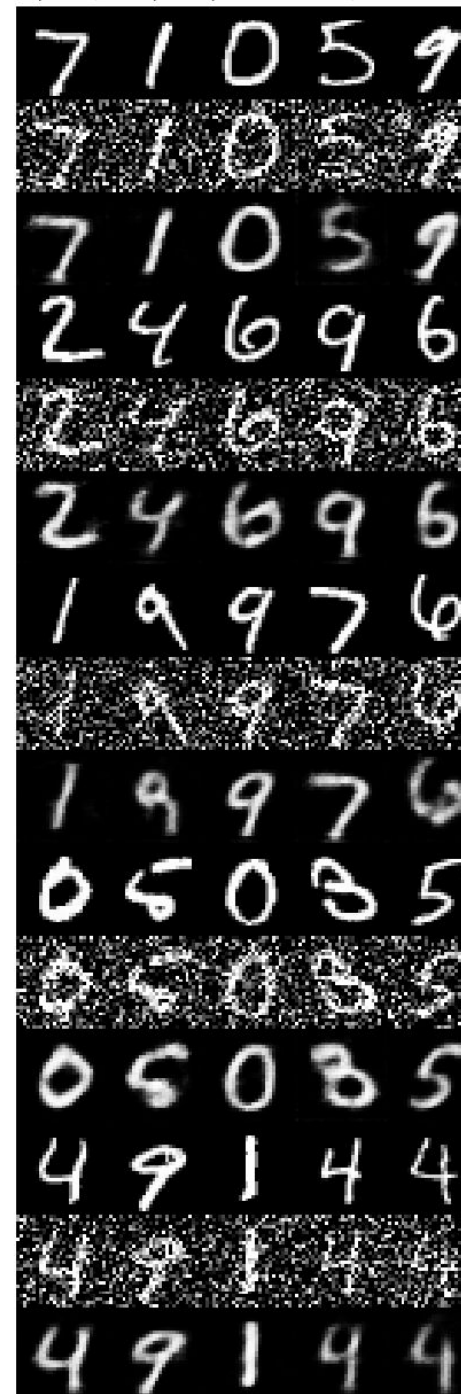


x



Demo

Original images: top rows, Corrupted Input: middle rows, Denoised Output: bottom rows



3715895

Generative: Colorization

- Self-Supervised Learning
- Transfer Learning
- Self-Supervised Learning Approaches
- Generative
 - Autoencoders
 - Colorization
 - Cross-Channel Prediction
 - Context-Encoders (Inpainting)
 - Image Super-Resolution
- Discriminative
 - Rotation
 - Jigsaw
 - Clustering
 - Contrastive Learning



Image Colorization



- *Image colorization*

- Efros' Group: Colorful Image Colorization
- **Training data:** Pairs of color and grayscale images.
- **Pretext task:** Predict the **colors** of the objects in grayscale images.
 - The model needs to understand the objects in images and paint them with a suitable color.
 - Right image: Learn that the sky is blue, cloud is white, and mountain is green.

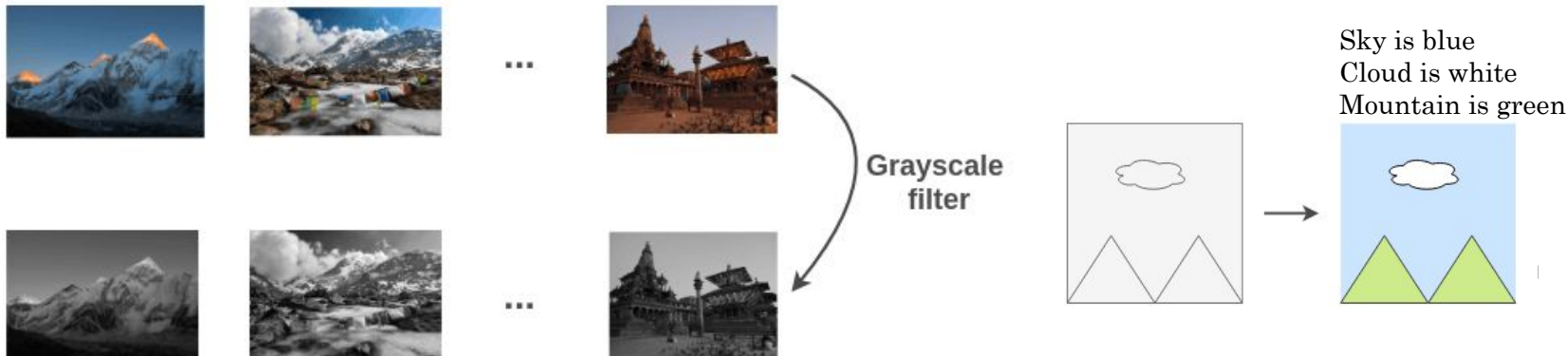


Image Colorization



- An encoder-decoder architecture with convolutional layers is used
 - ℓ_2 loss between the actual color image and the predicted colorized image is used for training.

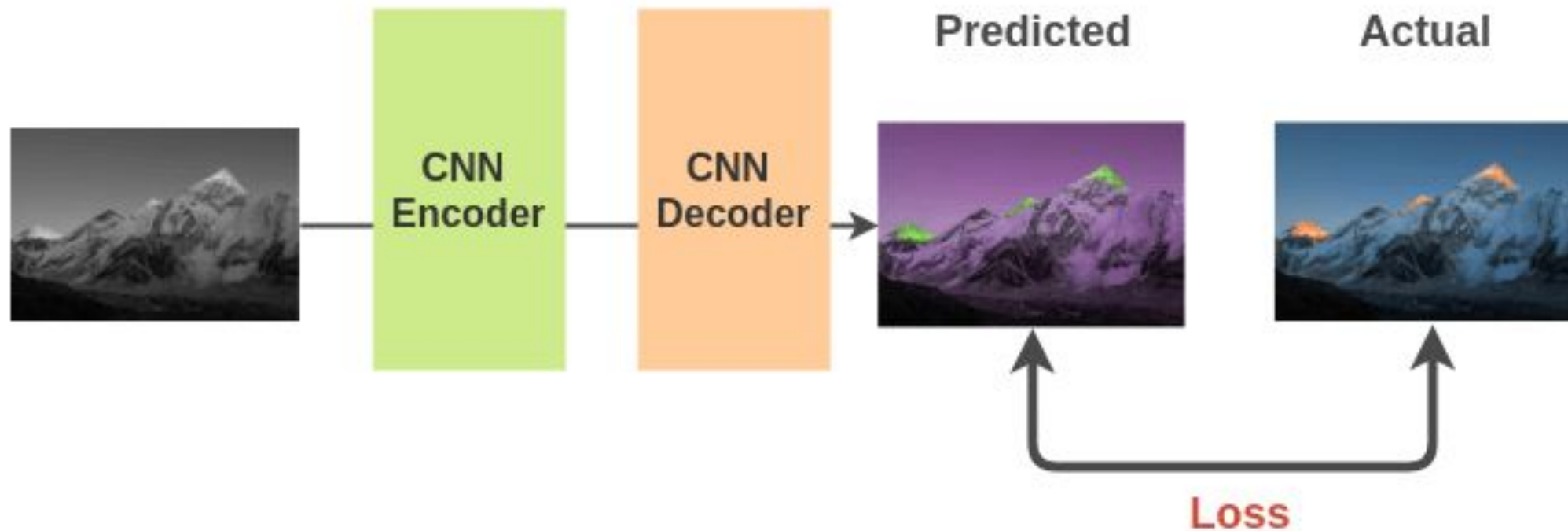
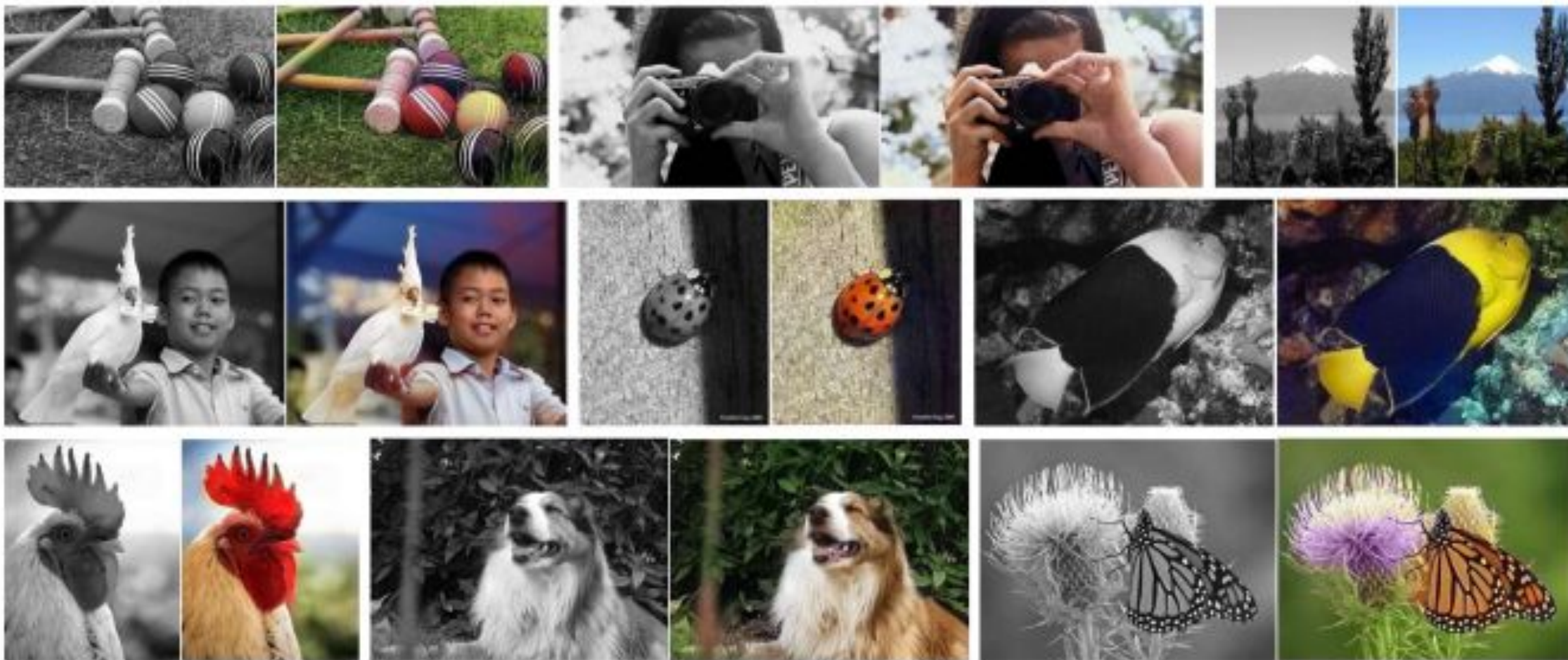


Image Colorization



3715895



Generative: Cross-Channel Prediction

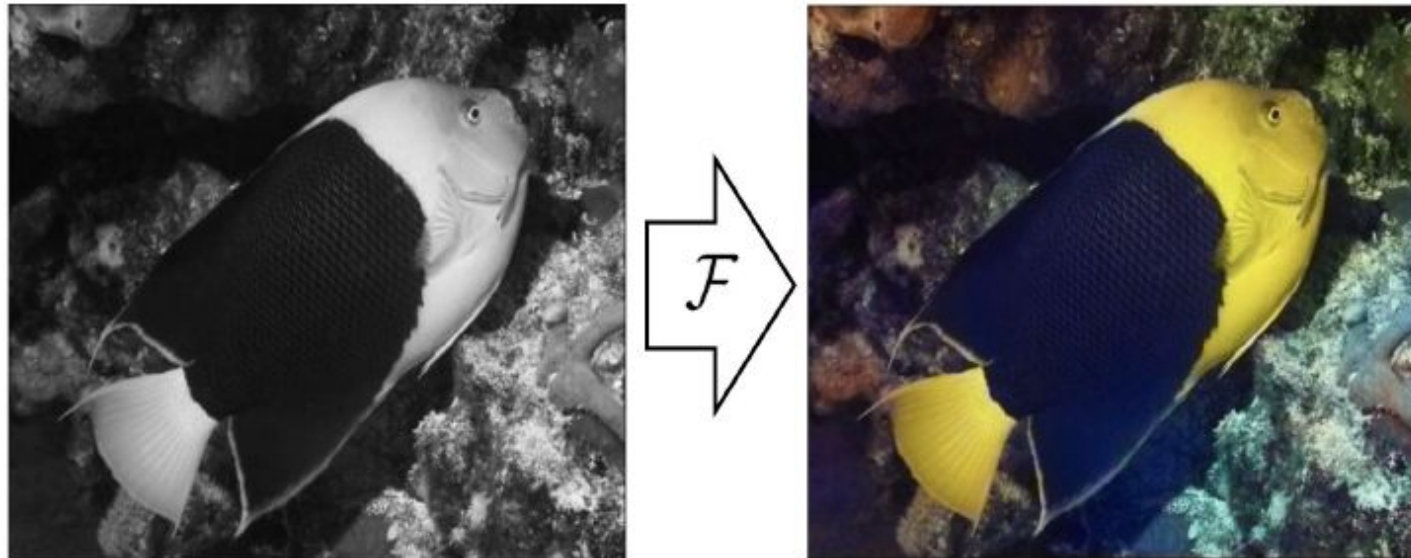
- Self-Supervised Learning
- Transfer Learning
- Self-Supervised Learning Approaches
- Generative
 - Autoencoders
 - Colorization
 - Cross-Channel Prediction
 - Context-Encoders (Inpainting)
 - Image Super-Resolution
- Discriminative
 - Rotation
 - Jigsaw
 - Clustering
 - Contrastive Learning



Cross-Channel Prediction



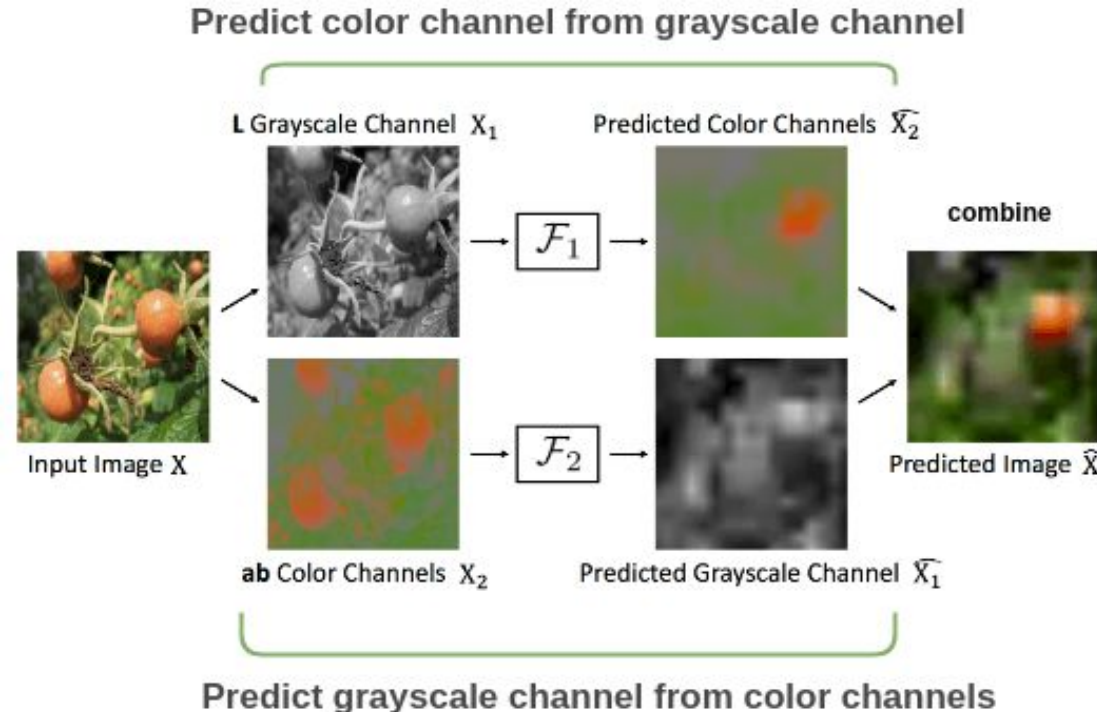
- *Split-brain autoencoder* or *cross-channel prediction*
 - Efros' Group: Split-Brain Autoencoders: Unsupervised Learning by Cross-Channel Prediction
- **Training data:** Remove some of the image color channels.
- **Pretext task:** Predict the **missing channel** from the other image channels.



Cross-Channel Prediction



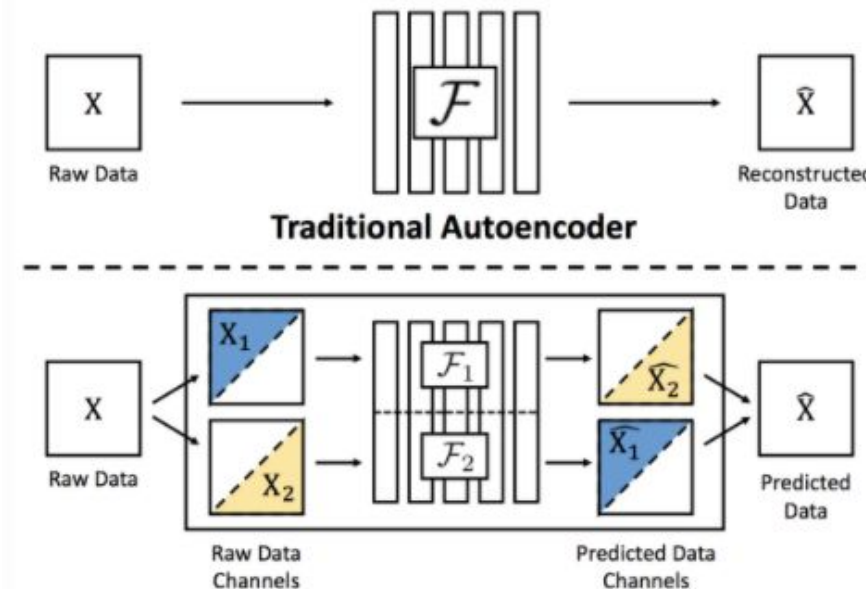
- The input image is split into grayscale and color channels.
 - Two encoder-decoders are trained: F_1 predicts the color channels from the gray channel, and F_2 predicts the gray channel from the color channels.
 - The two predicted images are combined to reconstruct the original image.
 - A loss function (e.g., cross-entropy) is used to minimize the reconstruction loss.



Cross-Channel Prediction



- The general model is shown below, in comparison to a traditional autoencoder.
 - Any combinations of image channels X_1 and X_2 can be used with this approach for training and reconstruction.
 - The reconstructed images are denoted with a “hat” notation ($\hat{X}_1, \hat{X}_2$).



Generative: Context-Encoders (Inpainting)

- Self-Supervised Learning
- Transfer Learning
- Self-Supervised Learning Approaches
- Generative
 - Autoencoders
 - Colorization
 - Cross-Channel Prediction
 - Context-Encoders (Inpainting)
 - Image Super-Resolution
- Discriminative
 - Rotation
 - Jigsaw
 - Clustering
 - Contrastive Learning



Context Encoders

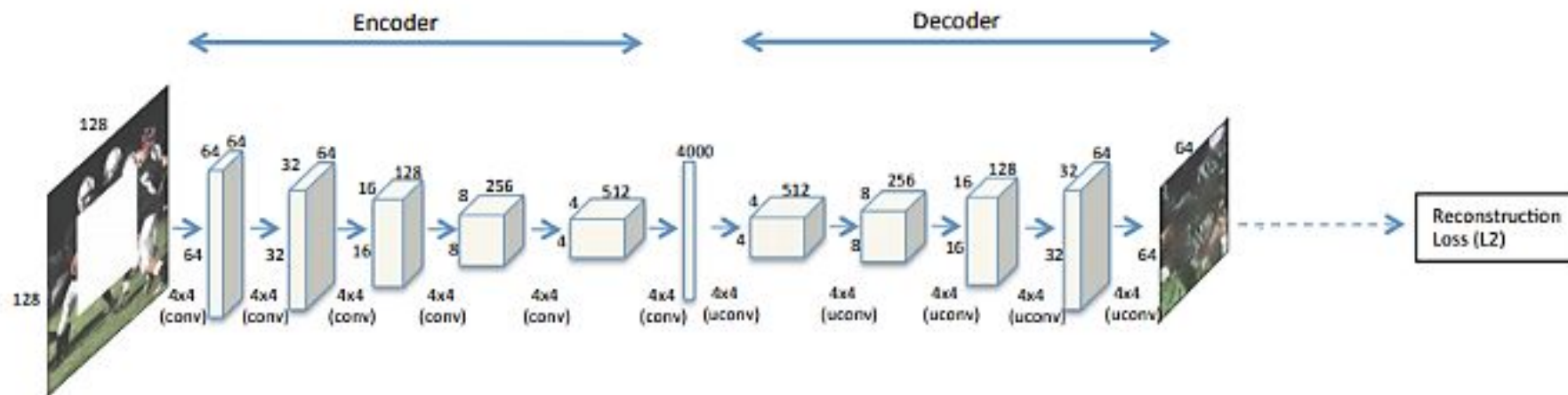
- *Predict missing pieces*, aka *context encoders*, or *inpainting*
 - Efros' Group: Context Encoders: Feature Learning by Inpainting
- **Training data:** Remove a random region in images.
- **Pretext task:** Fill in a **missing piece** in the image.
 - The model needs to understand the content of the entire image, and produce a plausible replacement for the missing piece



Context Encoders



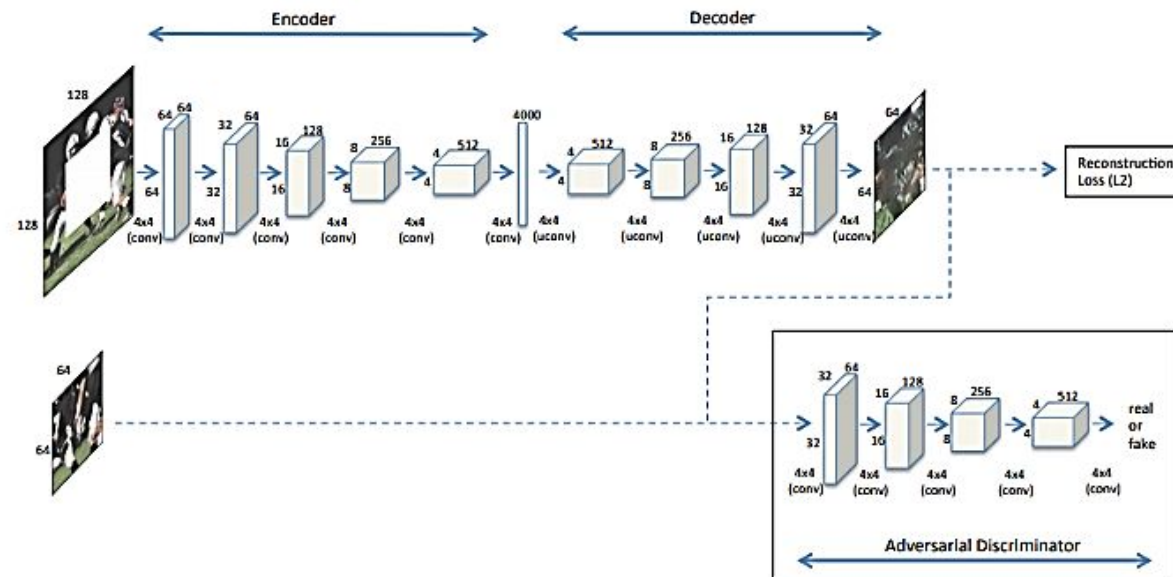
- The initially considered model uses an **encoder-decoder** architecture.
 - The encoder and decoder have multiple Conv layers, and a shared central fully-connected layer.
 - The output of the decoder is the reconstructed input image.
 - A Euclidean ℓ_2 distance is used as the reconstruction loss function $\mathcal{L}_{\text{rec}}$.



Context Encoders

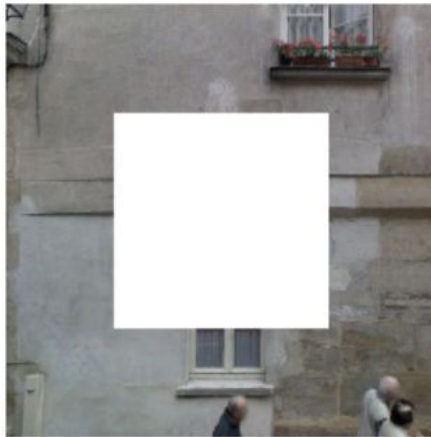


- The authors found that the reconstruction loss alone couldn't capture fine details in the missing region.
- Improvement was achieved by adding a GAN branch, where the generator of the GAN learns to reconstruct the missing piece
 - The overall loss function is a weighted combination of the reconstruction and the discriminator losses, i.e., $\mathcal{L} = \lambda_{\text{rec}}\mathcal{L}_{\text{rec}} + \lambda_{\text{gan}}\mathcal{L}_{\text{gan}}$



Context Encoders

- The joint loss function $\mathcal{L} = \lambda_{\text{rec}}\mathcal{L}_{\text{rec}} + \lambda_{\text{gan}}\mathcal{L}_{\text{gan}}$ resulted in improved prediction of the missing piece



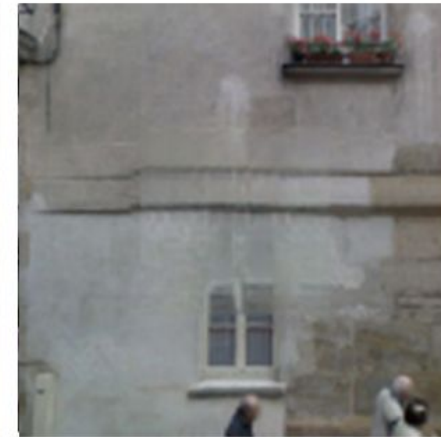
Input image



Encoder-decoder
with reconstruction
loss $\mathcal{L}_{\text{rec}}$



GAN with loss $\mathcal{L}_{\text{gan}}$

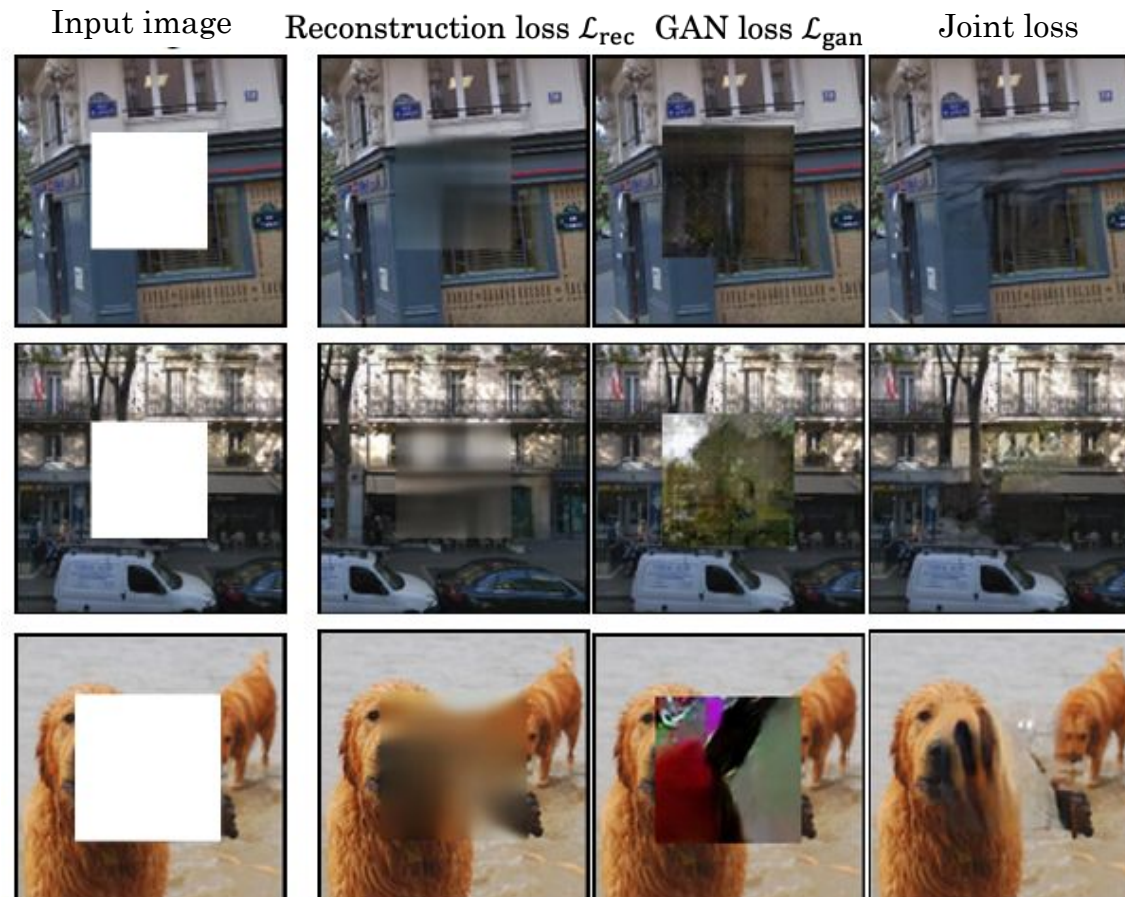


Joint loss
 $\mathcal{L} = \lambda_{\text{rec}}\mathcal{L}_{\text{rec}} +$
 $\lambda_{\text{gan}}\mathcal{L}_{\text{gan}}$

Context Encoders



- Additional examples comparing the used loss functions
 - The joint loss produces the most realistic images

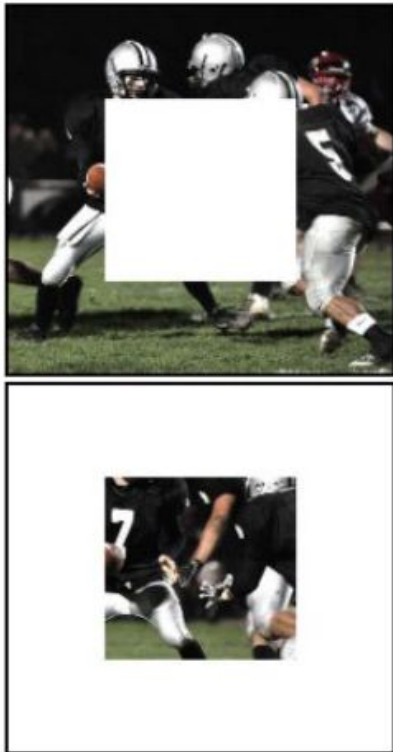


Context Encoders

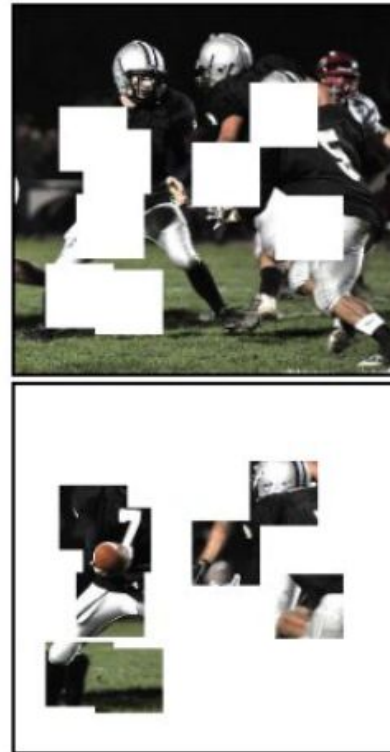


- The removed regions can have different shapes.
- Random region and random block masks outperformed the central region features

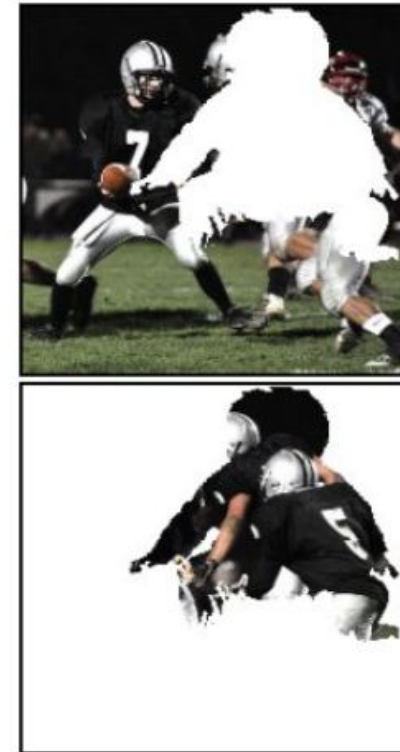
Central region



Random block



Random region





Context Encoders

- Evaluation on PASCAL VOC for several downstream tasks.
 - The learned features by the context encoder are not as good as supervised features but are comparable to other unsupervised methods, and perform better than randomly initialized models
 - E.g., over 10% improvement for segmentation over random initialization

Pretraining Method	Supervision	Pretraining time	Classification	Detection	Segmentation
ImageNet [26]	1000 class labels	3 days	78.2%	56.8%	48.0%
Random Gaussian	initialization	< 1 minute	53.3%	43.4%	19.8%
Autoencoder	-	14 hours	53.8%	41.9%	25.2%
Agrawal <i>et al.</i> [1]	egomotion	10 hours	52.9%	41.8%	-
Doersch <i>et al.</i> [7]	context	4 weeks	55.3%	46.6%	-
Wang <i>et al.</i> [39]	motion	1 week	58.4%	44.0%	-
Ours	context	14 hours	56.5%	44.5%	29.7%

Generative: Image Super Resolution

- Self-Supervised Learning
- Transfer Learning
- Self-Supervised Learning Approaches
- Generative
 - Autoencoders
 - Colorization
 - Cross-Channel Prediction
 - Context-Encoders (Inpainting)
 - Image Super-Resolution
- Discriminative
 - Rotation
 - Jigsaw
 - Clustering
 - Contrastive Learning



Image Super-Resolution



3715895

- *Image Super-Resolution*

- Ledig (2017) Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network
- **Training data:** Pairs of regular and down-sampled low-resolution images.
- **Pretext task:** Predict a **high-resolution image** that corresponds to a down-sampled low-resolution image.



...



Make 2x smaller

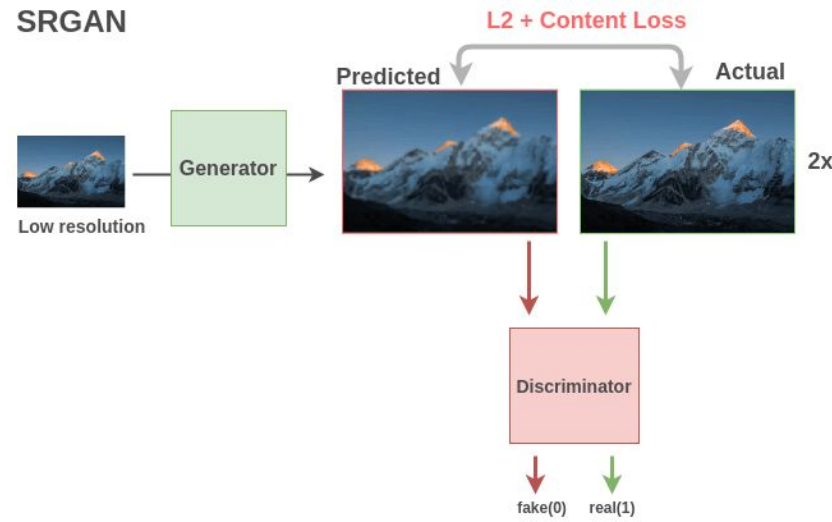


...

Image Super-Resolution



- SRGAN (Super-Resolution GAN) is a variant of GAN for producing super-resolution images
 - The **generator** takes a low-resolution image and outputs a high-resolution image using a fully-convolutional network.
 - The **discriminator**'s loss function combines L_2 and content loss to distinguish between real and generated (fake) super-resolution images.
 - **Content loss (part of the perceptual loss)** compares the features from the actual and predicted images.



Discriminative: Rotation

- Self-Supervised Learning
- Transfer Learning
- Self-Supervised Learning Approaches
- Generative
 - Autoencoders
 - Colorization
 - Cross-Channel Prediction
 - Context-Encoders (Inpainting)
 - Image Super-Resolution
- Discriminative
 - Rotation
 - Jigsaw
 - Clustering
 - Contrastive Learning

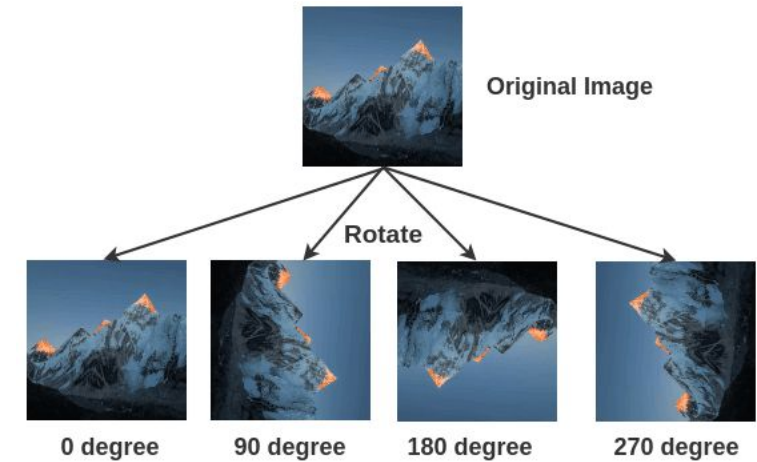


Image Rotation



- *Geometric transformation recognition*

- Gidaris (2018) - Unsupervised Representation Learning by Predicting Image Rotations
- **Training data:** Images rotated by a multiple of 90° at random.
 - This corresponds to four rotated images at 0° , 90° , 180° , and 270° .
- **Pretext task:** Train a model to **predict the rotation degree** that was applied
 - Therefore, it is a 4-class classification problem



Architecture for Geometric Transformation Recognition

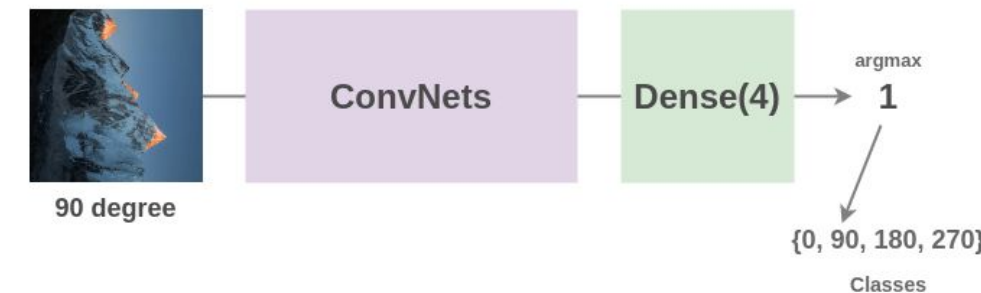


Image Rotation



- A single ConvNet model is used to predict one of the four rotations.
 - The model needs to understand the location and type of the objects in images to understand the rotation degree.

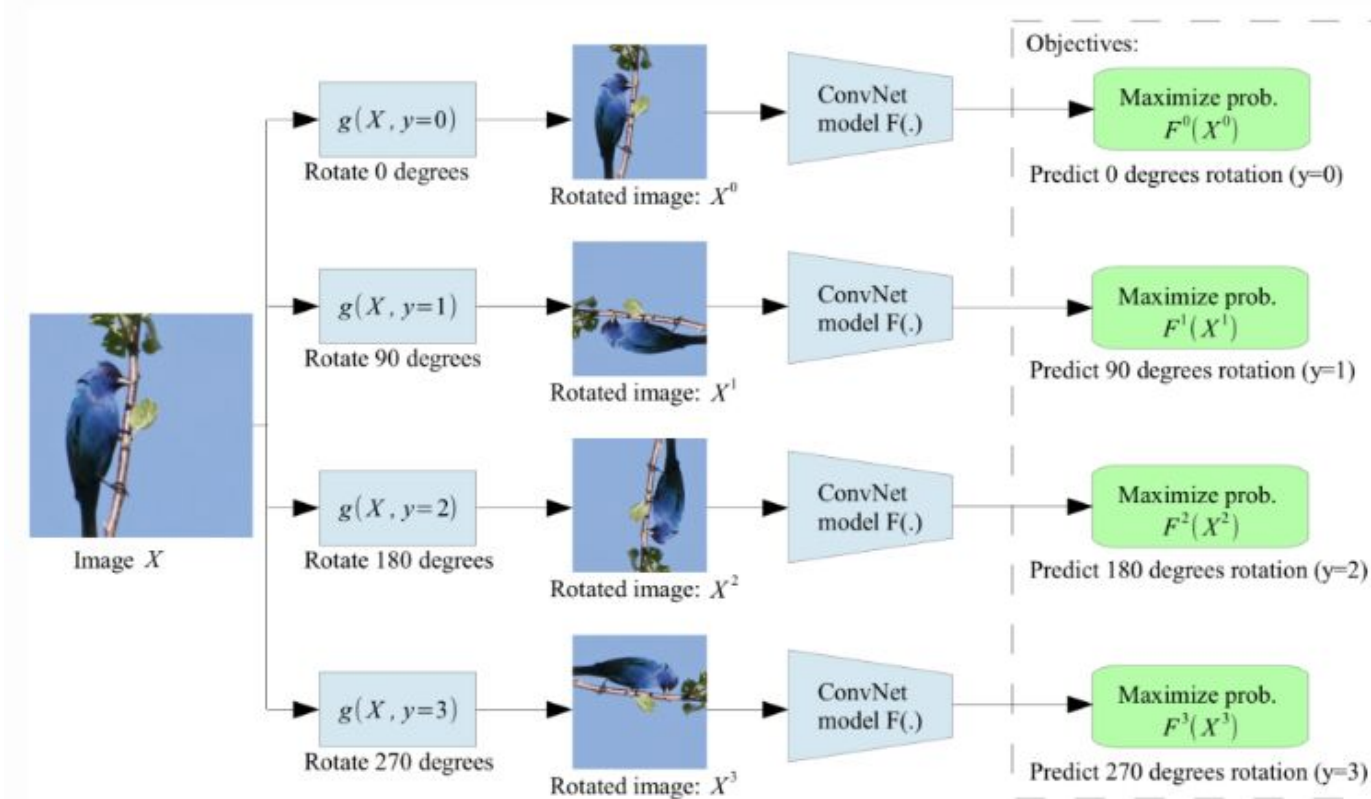




Image Rotation - Generalizability

# Rotations	Rotations	CIFAR-10 Classification Accuracy
4	$0^\circ, 90^\circ, 180^\circ, 270^\circ$	89.06
8	$0^\circ, 45^\circ, 90^\circ, 135^\circ, 180^\circ, 225^\circ, 270^\circ, 315^\circ$	88.51
2	$0^\circ, 180^\circ$	87.46
2	$90^\circ, 270^\circ$	85.52



ImageNet Top-1 Classification Results

Method	Conv4	Conv5
ImageNet labels from (Bojanowski & Joulin, 2017)	59.7	59.7
Random from (Noroozi & Favaro, 2016)	27.1	12.0
Tracking Wang & Gupta (2015)	38.8	29.8
Context (Doersch et al., 2015)	45.6	30.4
Colorization (Zhang et al., 2016a)	40.7	35.2
Jigsaw Puzzles (Noroozi & Favaro, 2016)	45.3	34.6
BIGAN (Donahue et al., 2016)	41.9	32.2
NAT (Bojanowski & Joulin, 2017)	-	36.0
(Ours) RotNet	50.0	43.8

With non-linear layers



Task Generalization

Method	Conv1	Conv2	Conv3	Conv4	Conv5
ImageNet labels	19.3	36.3	44.2	48.3	50.5
Random	11.6	17.1	16.9	16.3	14.1
Random rescaled Krähenbühl et al. (2015)	17.5	23.0	24.5	23.2	20.6
Context (Doersch et al., 2015)	16.2	23.3	30.2	31.7	29.6
Context Encoders (Pathak et al., 2016b)	14.1	20.7	21.0	19.8	15.5
Colorization (Zhang et al., 2016a)	12.5	24.5	30.4	31.5	30.3
Jigsaw Puzzles (Noroozi & Favaro, 2016)	18.2	28.8	34.0	33.9	27.1
BIGAN (Donahue et al., 2016)	17.7	24.5	31.0	29.9	28.0
Split-Brain (Zhang et al., 2016b)	17.7	29.3	35.4	35.2	32.8
Counting (Noroozi et al., 2017)	18.0	30.6	34.3	32.5	25.7
(Ours) RotNet	18.8	31.7	38.7	38.2	36.5

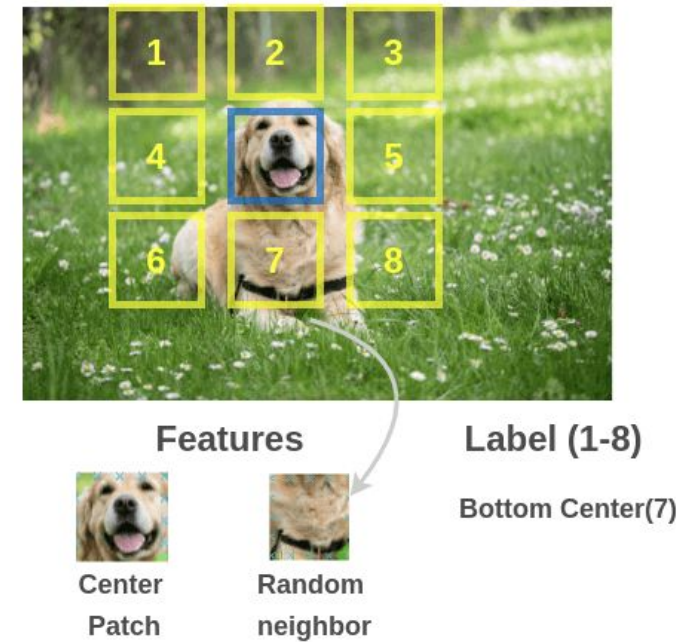
With linear layers



Relative Patch Position

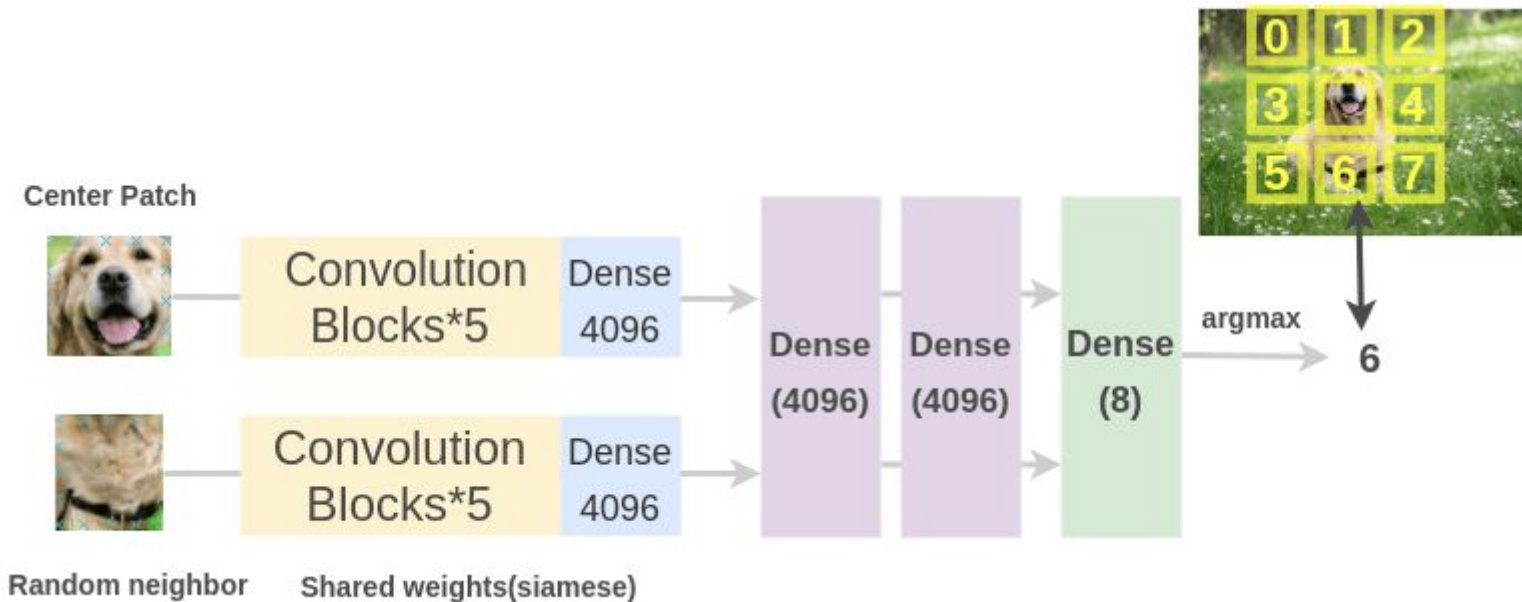
- *Relative patch position for context prediction*

- Efros' Group: Unsupervised Visual Representation Learning by Context Prediction
- **Training data:** Multiple **patches** extracted from images.
- **Pretext task:** Train a model to **predict the relationship between the patches**.
 - E.g., predict the relative position of the selected patch below (i.e., position # 7)
 - For the center patch, there are 8 possible neighbor patches (8 possible classes)



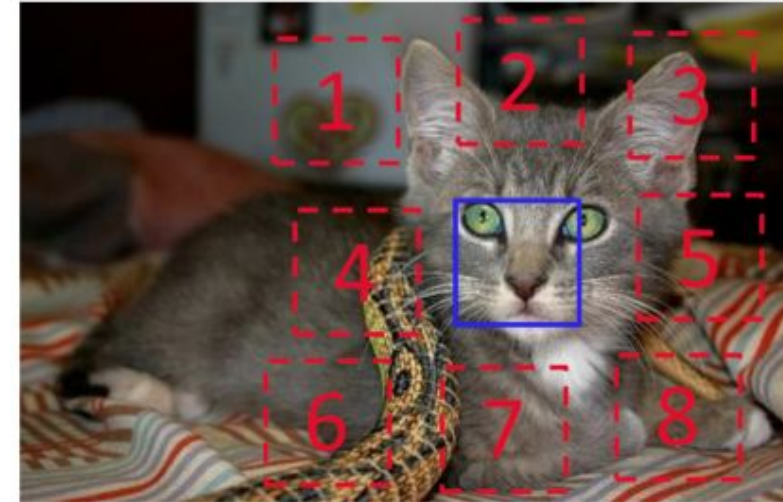
Relative Patch Position

- The patches are inputted into two ConvNets with shared weights
 - The learned features by the ConvNets are concatenated.
 - Classification is performed over 8 classes (8 possible neighbor positions)
- The model needs to understand the spatial context of images to predict the relative positions between the patches.



Relative Patch Position

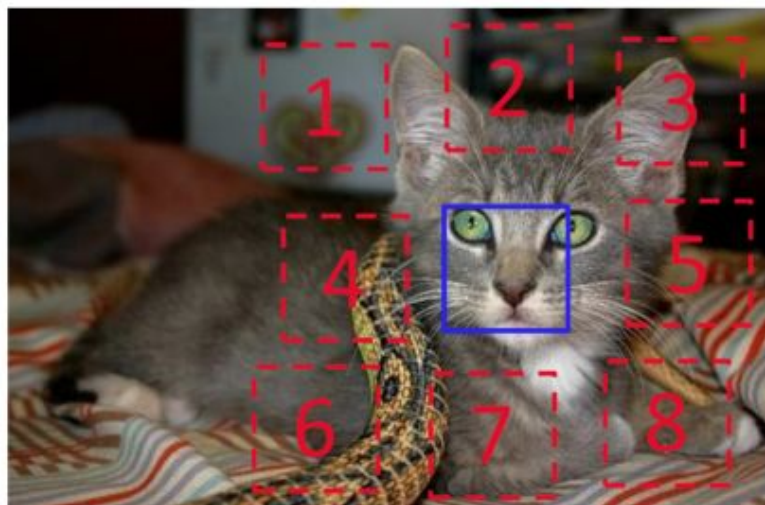
- Training patches are **sampled** as follows:
 - Randomly sample the first patch and consider it the middle of a 3×3 grid.
 - Sample from 8 neighboring locations of the first central patch (blue patch)/
- To avoid the model only catching low-level trivial information:
 - Add gaps between the patches.
 - Add small jitters to the positions of the patches.
 - Randomly down-sample some patches to reduced resolution, and then up-sample them.
 - Randomly drop 1 or 2 color channels for some patches.



Relative Patch Position



- For instance, predict the position of patch # 3 with respect to the central patch



- Input: two patches

$$X = (\text{patch}_1, \text{patch}_2)$$

- Prediction: $Y = 3$

Discriminative: Jigsaw

- Self-Supervised Learning
- Transfer Learning
- Self-Supervised Learning Approaches
- Generative
 - Autoencoders
 - Colorization
 - Cross-Channel Prediction
 - Context-Encoders (Inpainting)
 - Image Super-Resolution
- Discriminative
 - Rotation
 - Jigsaw
 - Clustering
 - Contrastive Learning



Image Jigsaw Puzzle



- *Predict patches position in a jigsaw puzzle*
 - Noroozi (2016) Unsupervised Learning of Visual Representations by Solving Jigsaw Puzzles
- **Training data:** 9 patches extracted in images (similar to the previous approach)
- **Pretext task:** Predict the **positions of all 9 patches** (9! options)
 - Instead of predicting the relative position of only 2 patches, this approach uses the grid of 3×3 patches and solves a jigsaw puzzle.

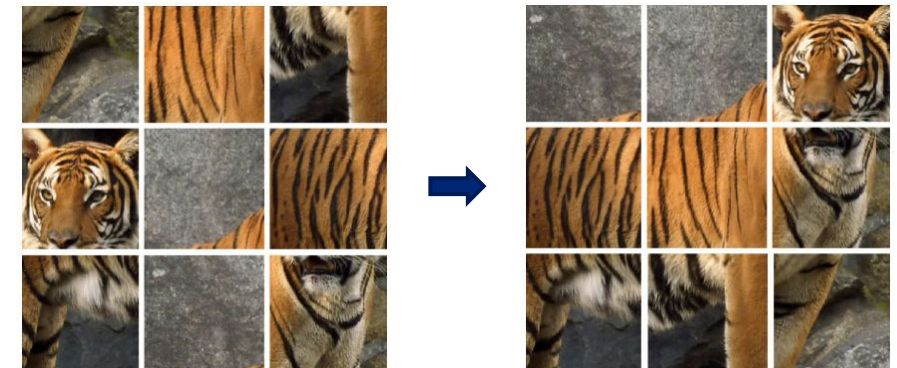
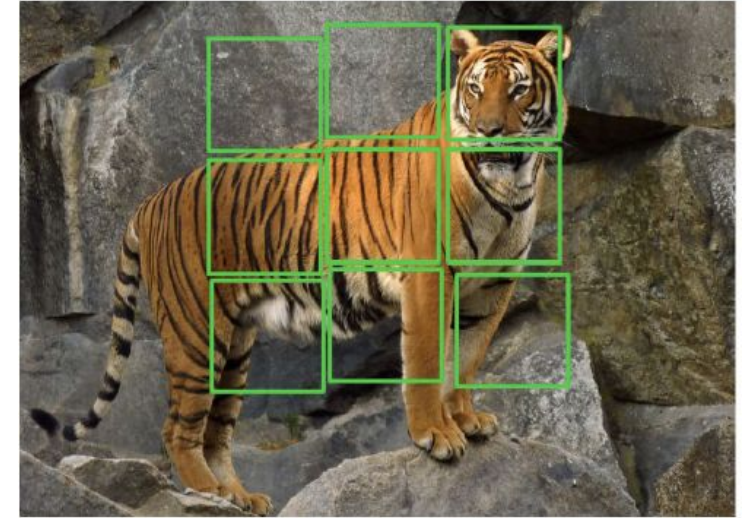
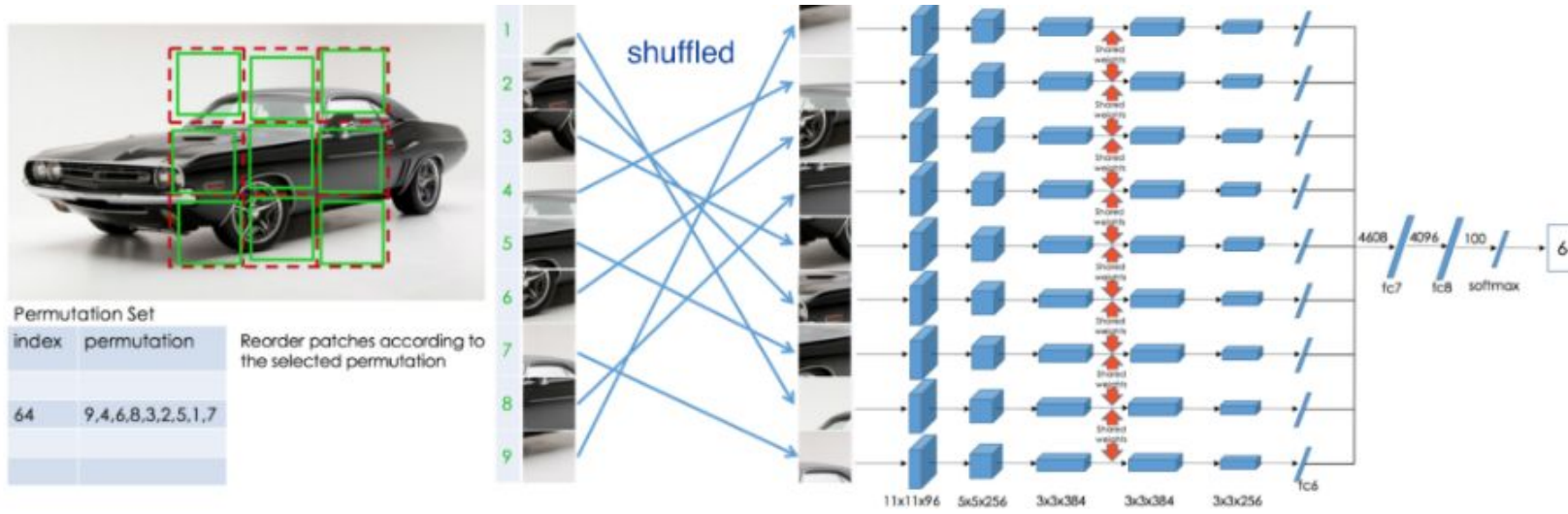


Image Jigsaw Puzzle



- A ConvNet model passes the individual patches through the same Conv layers that have shared weights.
- The features are combined and passed through fully-connected layers.
- Output is the positions of the patches
 - The patches are shuffled according to a set of 64 predefined permutations
 - Namely, for 9 patches, in total there are $9! = 362,880$ possible puzzles
 - The authors used a small set of 64 shuffling permutations with the highest hamming distance



Discriminative: Clustering

- Self-Supervised Learning
- Transfer Learning
- Self-Supervised Learning Approaches
- Generative
 - Autoencoders
 - Colorization
 - Cross-Channel Prediction
 - Context-Encoders (Inpainting)
 - Image Super-Resolution
- Discriminative
 - Rotation
 - Jigsaw
 - Clustering
 - Contrastive Learning



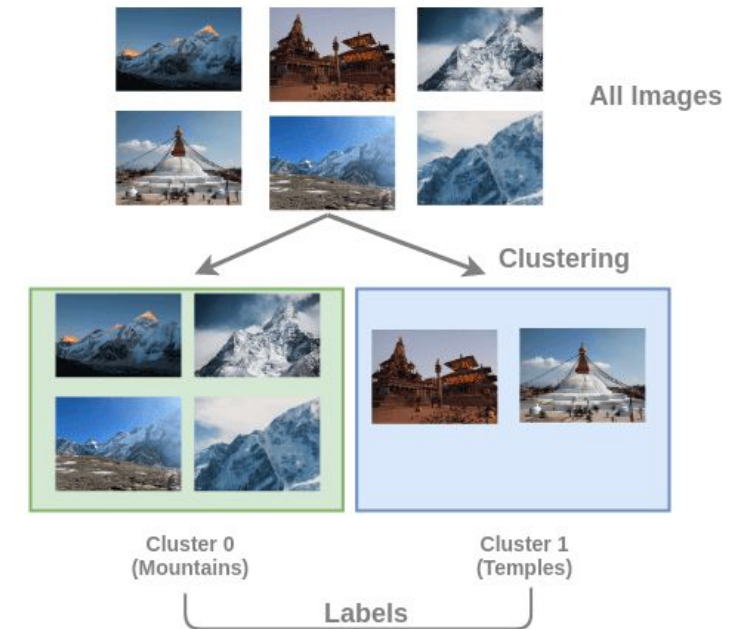
3715895

Deep Clustering



- *Deep clustering of images*

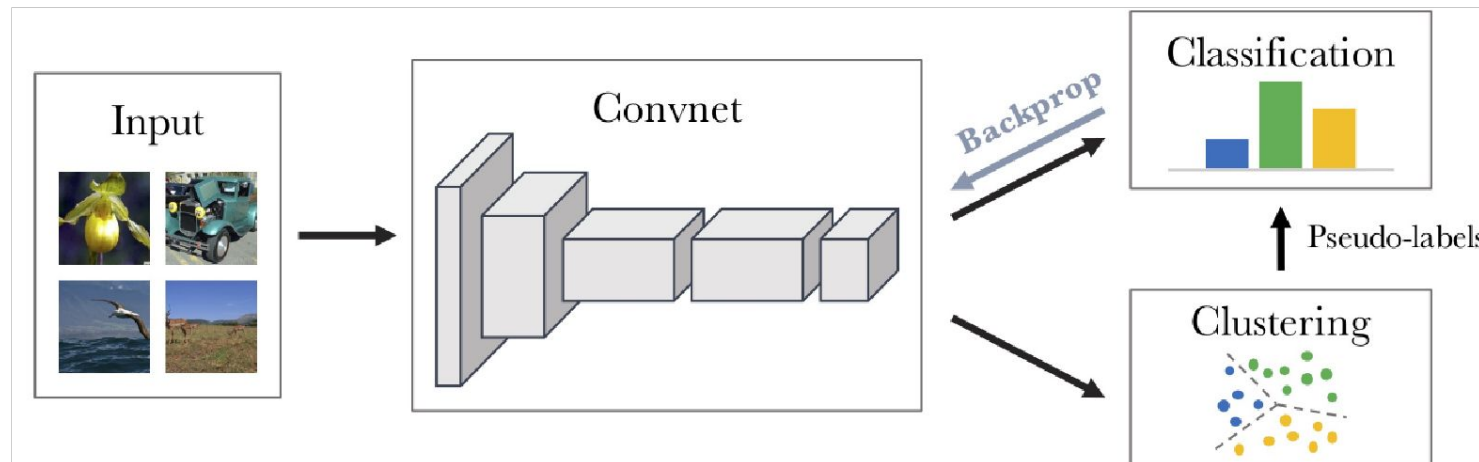
- Caron (2019) Deep Clustering for Unsupervised Learning of Visual Features
- **Training data:** Clusters of images based on the content
 - E.g., clusters on mountains, temples, etc.
- **Pretext task:** Predict the **cluster** to which an image belongs



Deep Clustering



- The architecture for SSL is called **deep clustering**.
 - The model treats each cluster as a separate class.
 - The output is the number of the cluster (i.e., cluster label) for an input image.
 - The authors used Kmeans for clustering.
- The model needs to learn the content in the images to assign them to the corresponding cluster



Contrastive Representation Learning

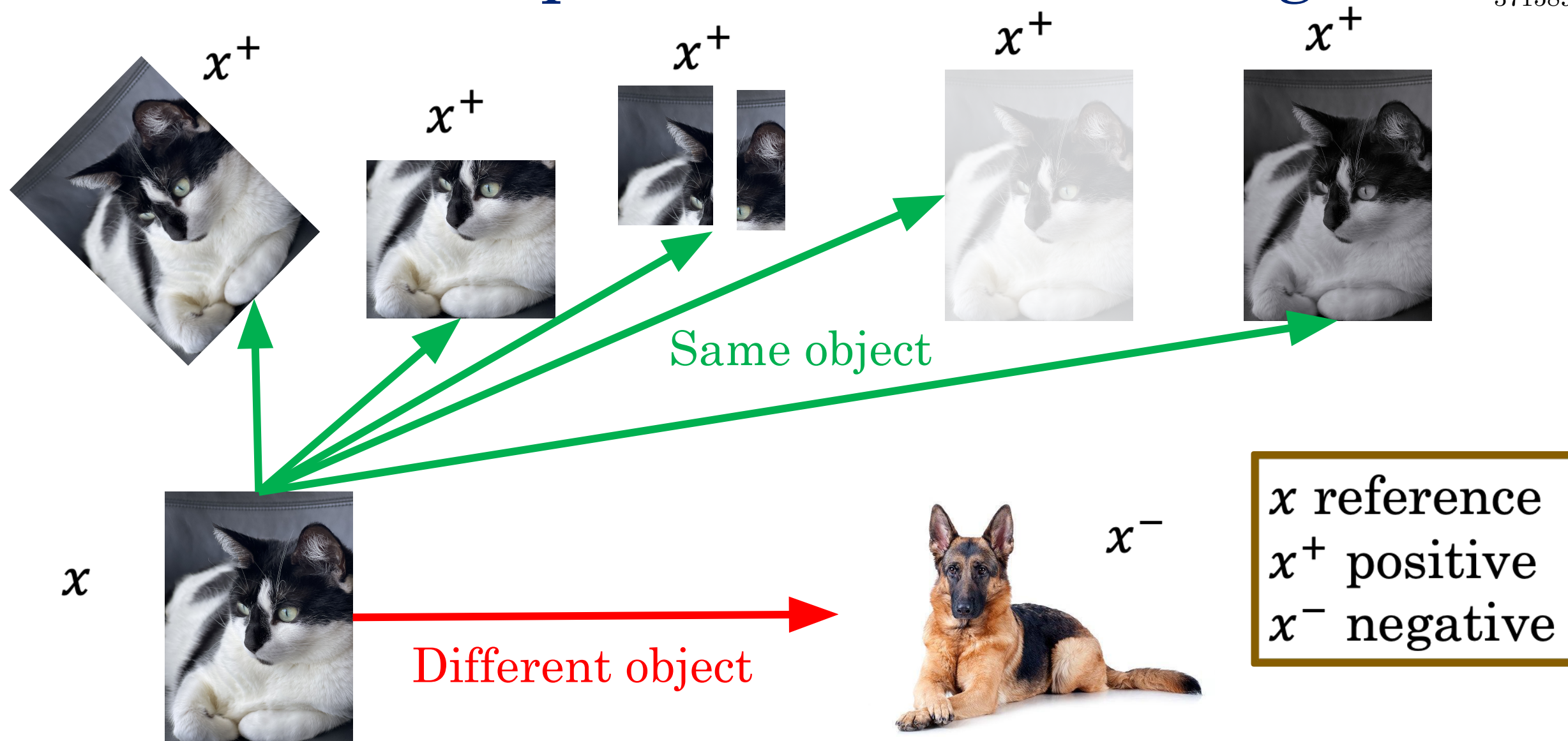
- Self-Supervised Learning
- Transfer Learning
- Self-Supervised Learning Approaches
- Generative
 - Autoencoders
 - Colorization
 - Cross-Channel Prediction
 - Context-Encoders (Inpainting)
 - Image Super-Resolution
- Discriminative
 - Rotation
 - Jigsaw
 - Clustering
 - Contrastive Learning



Contrastive Representation Learning



3715895





Formulation for Contrastive Learning

- What we want:

$$\textit{score}(f(x), f(x^+)) \gg \textit{score}(f(x), f(x^-))$$

x reference
 x^+ positive
 x^- negative

- Given a chosen score function, we aim to learn an encoder function f that yields high score for positive pairs (x, x^+) and low scores for negative pairs (x, x^-) .



Contrastive Loss

- Given 1 positive sample and N-1 negative samples:

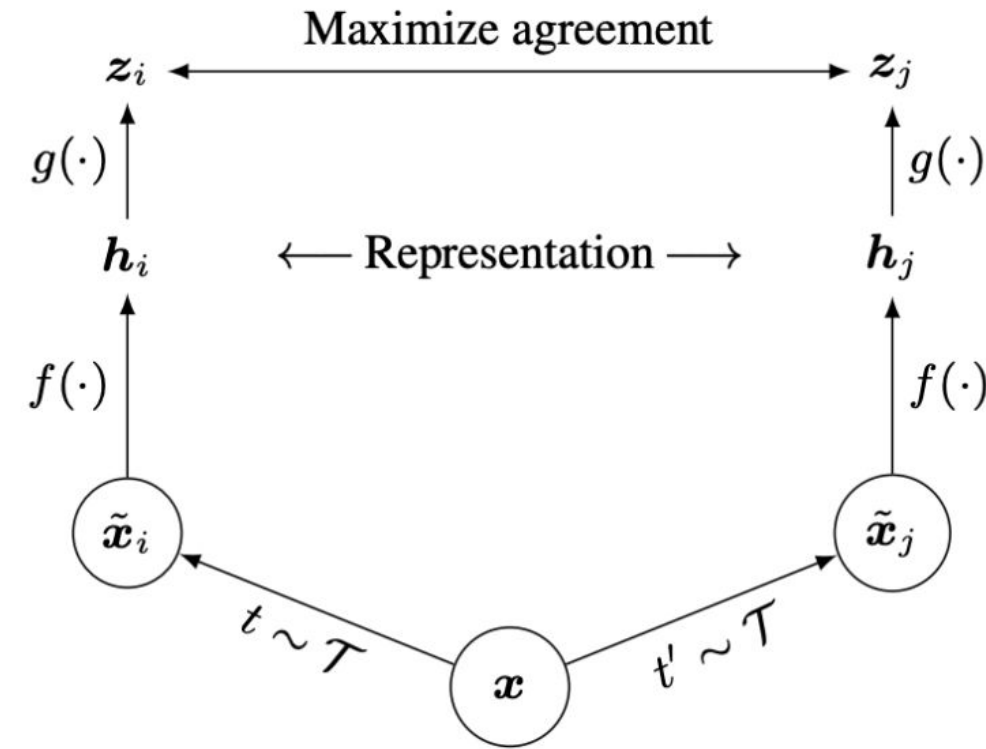
$$L = -\mathbb{E}_x \left[\log \frac{\exp s(f(x), f(x^+))}{\exp s(f(x), f(x^+)) + \sum_{j=1}^{N-1} \exp s(f(x), f(x^-))} \right]$$

This is similar to cross-entropy loss for a N-way softmax classifier.

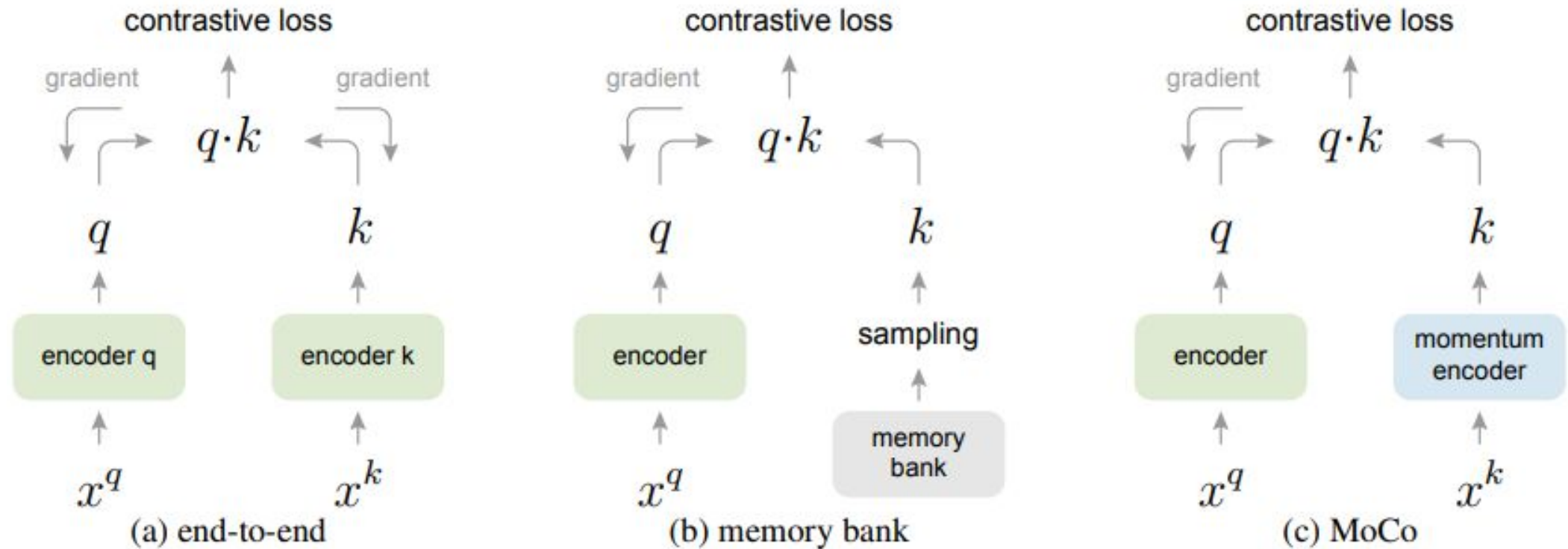
SimCLR: A Simple Framework for Contrastive Learning



- Cosine similarity as the score function
- Use a projection network $g(\cdot)$ to project features to a space where contrastive learning is applied.
- Generate positive samples through data augmentation:
 - Random cropping, random color distortion, and random blur.



Momentum Contrastive Learning (MoCo)



Decouples negative sample size from minibatch size; allows large batch training without TPUs.

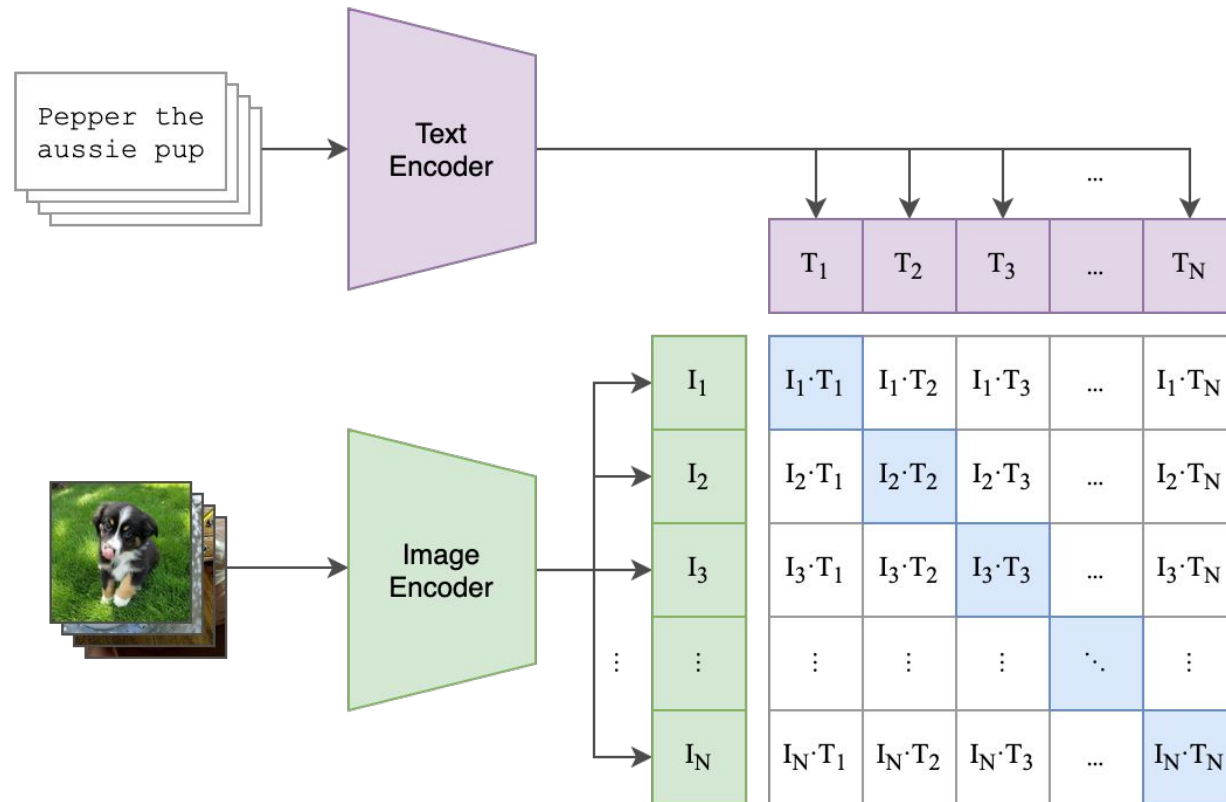
<https://arxiv.org/abs/1911.05722>

Contrastive Language Image Pre-Training (CLIP)

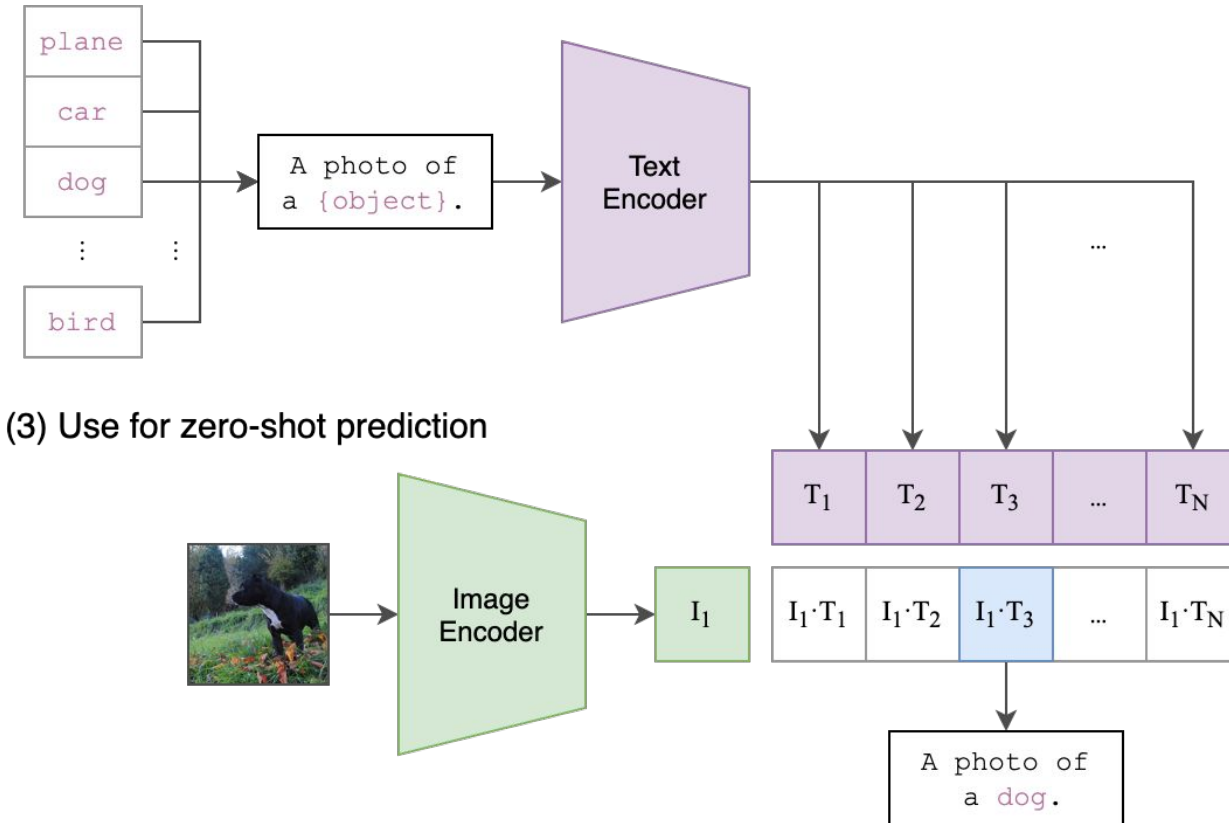


3715895

(1) Contrastive pre-training



(2) Create dataset classifier from label text



Lecture 24

Self-Supervised Learning

Reading: Partially covered in Chapter 19 of Bishop Deep Learning Textbook